

ЕТАП 6 ОТ 11 · РЪКОВОДСТВО

Звуци الأذان

Всяка молитва има свой звук: пълен езан за алармата на Android и кратък откъс (до 30 сек.) за iPhone и за известието без „Аларми и напомняния“. Известията имат отделен списък – звукът на телефона, 22 звъна и свои до 10 сек. Всичко се преслушва с ►, а всеки може да добави и свой звук от телефона.

4 записа на езан

22 звука за известия

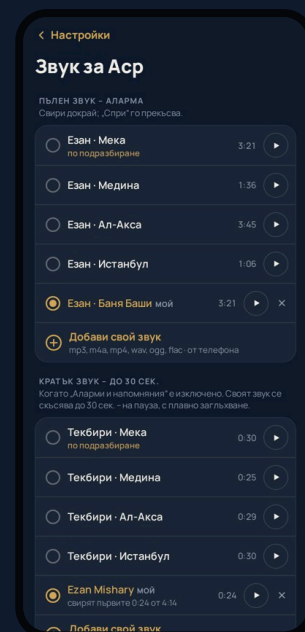
expo-audio

expo-document-picker

Kotlin · Swift

Jest – 218 теста

14 файла



Какво е решено в този етап

Два вида звуци: пълни (само алармата на Android) и кратки до 30 сек. (iPhone; Android без точни аларми)
· езан Мека, Медина, Ал-Акса, Истанбул + текбирите от тях · свой звук: пълен до 15 мин. / 60 MB, кратък – приложението само реже откъс до 30 сек. на пауза, с плавно заглъхване · звук на известията: по подразбиране звукът на телефона, свой – само до 10 сек. · мелодиите отпаднаха · лицензът на своите звуци е на потребителя.

СЪДЪРЖАНИЕ

01 Какво прави етапът

02 Нови пакети и нов build

03 Файловете на звуците

04 Каталогът и изборът

05 Свои звуци и преслушване

06 Известията и алармата

07 Екраните

08 Тестове

09 Проверка и какво остава

Стъпка 1

Какво прави етапът

Кой звук къде свири:

	ANDROID	IPHONE
Камбанка „аларма“	пълният звук – истинската аларма (етап 5)	краткият звук (до 30 сек.) като звук на известието
Без „Alarms & reminders“	краткият звук като звук на известието	—
Камбанка „известие“ и напомняне	звукът на известията	звукът на известията
Свой звук за алармата	пълнен – до 15 мин. / 60 MB; кратък – откъс до 30 сек.	откъс до 30 сек. (.caf)
Свой звук за известията	до 10 сек.; по-дълъг не се добавя	до 10 сек.; по-дълъг не се добавя

Дърво на проекта

```
Настройки → Звуци                                src/components/settings/SoundsGroup.tsx
  Фаджр · Зухр · Аср · Магриб · Иша ▶ > + Звук на известията ▶ >
    | тап
    ▼
Избор на звук                                       src/app/sound.tsx
  по подразбиране най-отгоре · вградени · МОЙ x · „+ Добави свой звук“
  „Същият звук за всички молитви“
    |
    ▼
useSounds (запазва се в телефона)                  src/store/sounds.ts
  | промяна → пренасрочване
  ▼
rescheduleNotifications()                          src/services/notifications.ts
  |— Android, аларма → setAlarms(): res/raw или свой файл
  |— известия → канал за всеки звук („ch4-n-system-v“, „ch4-a-takbir_makkah-v“);
    на iPhone – файлът на звука
```

Стъпка 2

Нови пакети и нов build

Три пакета с native код, нови звуци в `app.json` и нов config plugin – нужен е prebuild и нов build.

Инсталиране, prebuild и нов build

```
npx expo install expo-audio expo-document-picker expo-file-system
npx expo prebuild --platform android --clean
npx expo run:android
```

- **expo-audio** – преслушване на iPhone. В `app.json` без микрофон и без фоново възпроизвеждане.
- **expo-document-picker** – изборът на файл от телефона (без разрешения).
- **expo-file-system** – копие на своя пълен звук в паметта на приложението.
- `blockedPermissions` маха ненужните `READ/WRITE_EXTERNAL_STORAGE` и `RECORD_AUDIO`.

- **Кога е нужен prebuild:** при всяка промяна в `app.json` (звуци, плъгини, разрешения). Без него новите звуци не стигат до `res/raw` и вместо тях свири резервният – езанът. При промени само в JavaScript е достатъчен refresh.
- На iPhone native модулят вече има и Swift част – при следващия iOS build: `npx expo run:ios` (или EAS).

Стъпка 3

Файловете на звуците

Два списъка – защото пълните звуци не трябва на iPhone.

Дърво на проекта

```
assets/sounds/full/                ← пълни – само Android (plugins/withFullSounds.js → res/raw)
  adhan_makkah.mp3    3:21          mp3 64 kbps моно, изчистени, -16 LUFS
  adhan_madinah.mp3   1:36
  adhan_aqsa.mp3      3:45
  adhan_istanbul.mp3  1:06
assets/sounds/                  ← кратки (wav, моно, 22 kHz) – expo-notifications → Android и iPhone
  takbir_makkah.wav · takbir_madinah.wav · takbir_aqsa.wav · takbir_istanbul.wav
                                ← началните текбири, отрязани на пауза (до 30 сек.)
  ezan_chime.wav („Сигнал“) · chime_soft.wav („Тих звън“)
  notify_*.wav              ← 20 звука за известия (Pixabay), до 10 сек.
```

Правилото за рязане (едно и също за вградените, за Android и за iPhone): тишината в началото се маха; ако звукът е до 30 сек. – остава цял; иначе се реже на *последната истинска пауза* (поне 150 ms тихо) между 15-ата и 30-ата сек. с 0,5 сек. заглъхване – никога в средата на дума. Без пауза – на 30-ата сек. с 3 сек. заглъхване.

withFullSounds.js plugins

- Малък config plugin: копира `assets/sounds/full/*` в `res/raw` на Android – откъдето ги пуска алармата. Проверява имената (само a-z, 0-9, _).

plugins/withFullSounds.js

30 реда

```
/**
 * Пълните звуци на алармата (целият езан, мелодиите) – само за Android.
 * Копира assets/sounds/full/* в android/app/src/main/res/raw, откъдето ги пуска
 * native алармата (AlarmService). На iPhone не трябва – там звукът на известие е до 30 сек.,
 * затова не са в списъка на expo-notifications (иначе ще влязат и в приложението за iPhone).
 */
const { withDangerousMod } = require('expo/config-plugins');
const fs = require('fs');
const path = require('path');

module.exports = function withFullSounds(config) {
  return withDangerousMod(config, [
    'android',
    async (cfg) => {
      const root = cfg.modRequest.projectRoot;
      const from = path.join(root, 'assets', 'sounds', 'full');
      const to = path.join(cfg.modRequest.platformProjectRoot, 'app', 'src', 'main', 'res', 'raw');
      if (fs.existsSync(from)) {
        fs.mkdirSync(to, { recursive: true });
        for (const name of fs.readdirSync(from)) {
```

```

    if (!/^[a-z0-9_]+\.(mp3|m4a|wav|ogg)$/.test(name)) {
      throw new Error(`withFullSounds: невалидно име на звук „${name}“ (само a-z, 0-9, _)`);
    }
    fs.copyFileSync(path.join(from, name), path.join(to, name));
  }
}
return cfg;
},
]);
};

```

app.json корен на проекта

- Кратките звуци в expo-notifications, expo-audio без микрофон, `./plugins/withFullSounds`, `blockedPermissions`.

app.json

137 реда

```

{
  "expo": {
    "name": "Adhan",
    "slug": "adhan",
    "version": "1.0.0",
    "orientation": "portrait",
    "icon": "./assets/icon.png",
    "userInterfaceStyle": "dark",
    "ios": {
      "supportsTablet": false,
      "bundleIdentifier": "com.ilkoadamov.adhan",
      "infoPlist": {
        "CFBundleAllowMixedLocalizations": true
      }
    },
    "android": {
      "adaptiveIcon": {
        "backgroundColor": "#0F1B2D",
        "foregroundImage": "./assets/android-icon-foreground.png",
        "backgroundImage": "./assets/android-icon-background.png",
        "monochromeImage": "./assets/android-icon-monochrome.png"
      },
      "predictiveBackGestureEnabled": false,
      "package": "com.ilkoadamov.adhan",
      "permissions": [
        "android.permission.SCHEDULE_EXACT_ALARM"
      ],
      "blockedPermissions": [
        "android.permission.READ_EXTERNAL_STORAGE",
        "android.permission.WRITE_EXTERNAL_STORAGE",
        "android.permission.RECORD_AUDIO"
      ]
    },
    "web": {
      "favicon": "./assets/favicon.png"
    },
    "plugins": [
      "expo-router",
      [
        "expo-splash-screen",
        {
          "backgroundColor": "#0F1B2D",

```

```

    "image": "./assets/splash-icon.png",
    "imageWidth": 160
  }
],
"expo-font",
[
  "expo-build-properties",
  {
    "android": {
      "cmakeVersion": "3.31.6"
    }
  }
],
[
  "expo-localization",
  {
    "supportedLocales": {
      "ios": [
        "en",
        "bg"
      ],
      "android": [
        "en",
        "bg"
      ]
    }
  }
],
[
  "expo-location",
  {
    "locationWhenInUsePermission": "Adhan uses your location to show the prayer times for the place
where you are.",
    "locationAlwaysAndWhenInUsePermission": false,
    "locationAlwaysPermission": false,
    "motionUsagePermission": false
  }
],
"expo-background-task",
[
  "expo-notifications",
  {
    "icon": "./assets/notification-icon.png",
    "color": "#D4A857",
    "sounds": [
      "./assets/sounds/ezan_chime.wav",
      "./assets/sounds/chime_soft.wav",
      "./assets/sounds/takbir_makkah.wav",
      "./assets/sounds/takbir_madinah.wav",
      "./assets/sounds/takbir_aqsa.wav",
      "./assets/sounds/takbir_istanbul.wav",
      "./assets/sounds/notify_airport.wav",
      "./assets/sounds/notify_popup.wav",
      "./assets/sounds/notify_right.wav",
      "./assets/sounds/notify_modern.wav",
      "./assets/sounds/notify_magical.wav",
      "./assets/sounds/notify_chime_bright.wav",
      "./assets/sounds/notify_doorbell.wav",
      "./assets/sounds/notify_celestial.wav",
      "./assets/sounds/notify_bell_ding.wav",
      "./assets/sounds/notify_crisp_ding.wav",
    ]
  }
]

```

```

        "./assets/sounds/notify_shimmer.wav",
        "./assets/sounds/notify_smoke_alarm.wav",
        "./assets/sounds/notify_swoop.wav",
        "./assets/sounds/notify_retro.wav",
        "./assets/sounds/notify_robot.wav",
        "./assets/sounds/notify_three_beep.wav",
        "./assets/sounds/notify_machine_beep.wav",
        "./assets/sounds/notify_water_drop.wav",
        "./assets/sounds/notify_hang_up.wav",
        "./assets/sounds/notify_standard.wav"
    ]
}
],
[
    "expo-audio",
    {
        "microphonePermission": false,
        "recordAudioAndroid": false,
        "enableBackgroundPlayback": false,
        "enableBackgroundRecording": false
    }
],
"./plugins/withFullSounds"
],
"scheme": "adhan",
"backgroundColor": "#0F1B2D",
"experiments": {
    "typedRoutes": true
},
"locales": {
    "bg": "./locales/bg.json",
    "en": "./locales/en.json"
}
}
}

```

Стъпка 4

Каталогът и изборът

Чиста логика – с тестове.

sounds.ts src/domain

- Вградените звуци: 4 пълни, 4 откъса, „Звукът на телефона“, „Сигнал“, „Тих звън“ и 20-те звука за известия.
- По подразбиране: `adhan_makkah`, `takbir_makkah`, за известията – `system`.
- `fitsUse` – кой звук е за алармата и кой за известията; `findSound` – изтрит или неподходящ звук → звукът по подразбиране.
- `validateUpload` – 15 мин. / 60 MB за пълнен, 10 сек. за звук на известията.

src/domain/sounds.ts

295 реда

```

import type { PrayerId } from './prayers';

/**
 * Звуците на Езан (етап 6).
 *

```

```

* Два вида:
* - 'full' - пълни звуци (целият езан, мелодии с всякаква дължина). Свири ги само истинската
* аларма на Android (етап 5). Файловете са в assets/sounds/full → res/raw.
* - 'short' - кратки звуци до 30 сек. Свирият като звук на известие: на iPhone (там звукът на
* известие е най-много 30 сек.) и на Android без „Аларми и напомнания“.
* Файловете са в assets/sounds (wav) - в списъка на expo-notifications в app.json.
*
* Потребителят може да добави и свои звуци (виж src/services/sounds.ts): на Android - пълни,
* на iPhone - кратки (до 30 сек.).
*/

export type SoundKind = 'full' | 'short';
export type SoundCategory = 'adhan' | 'takbir' | 'chime' | 'custom';

/** Специалният „файл“ на звука на телефона (системният звук за известия). */
export const SYSTEM_SOUND = 'system';
/** Своят звук за известия - до 10 сек.; по-дълъг не се добавя. */
export const NOTIFY_MAX_SEC = 10;

/** На iPhone звукът на известие може да е най-много 30 сек. - иначе iOS пуска стандартния. */
export const SHORT_MAX_SEC = 30;
/** Своят пълен звук - до 15 мин. (алармата спира предпазно след 15 мин.) и до 60 MB. */
export const FULL_MAX_SEC = 15 * 60;
export const MAX_UPLOAD_BYTES = 60 * 1024 * 1024;

export interface SoundDef {
  id: string;
  kind: SoundKind;
  category: SoundCategory;
  names: { bg: string; en: string };
  /**
   * Вграден: името на файла без разширение („adhan_makkah“ - res/raw на Android, а за кратките
   * и „takbir_makkah.wav“ в приложението за iPhone).
   * Свой: пълният път до файла (file:///...) - на Android; на iPhone - името в Library/Sounds.
   */
  file: string;
  /** Разширението на вградения кратък звук (за expo-notifications). */
  ext?: string;
  durationSec: number;
  /** Свой звук на iPhone, скъсен до 30 сек.: колко е бил. */
  originalSec?: number;
  custom?: boolean;
  /** Свой звук: за алармата или за известията. */
  use?: SoundUse;
}

/** Вграден звук за известията: assets/sounds/notify_<key>.wav. */
function notifySound(key: string, bg: string, en: string, durationSec: number): SoundDef {
  return {
    id: `notify_${key}`,
    kind: 'short',
    category: 'chime',
    names: { bg, en },
    file: `notify_${key}`,
    ext: 'wav',
    durationSec,
  };
}

/**
 * Вградените звуци. Записите на езан Мека, Медина и Ал-Акса са от колекцията на

```

* PrayTimes.org, Истанбул – източникът да се уточни; изчистени, с изравнена сила (-16 LUFS);
* откъсите с текбири са отрязани на пауза (виж docs/PLAN.md – произходът им
* трябва да се потвърди преди публикуване).
*/

```
export const BUILTIN_SOUNDS: readonly SoundDef[] = [  
  // ---- пълни (само Android)  
  {  
    id: 'adhan_makkah',  
    kind: 'full',  
    category: 'adhan',  
    names: { bg: 'Езан · Мека', en: 'Adhan · Makkah' },  
    file: 'adhan_makkah',  
    durationSec: 201,  
  },  
  {  
    id: 'adhan_madinah',  
    kind: 'full',  
    category: 'adhan',  
    names: { bg: 'Езан · Медина', en: 'Adhan · Madinah' },  
    file: 'adhan_madinah',  
    durationSec: 96,  
  },  
  {  
    id: 'adhan_aqsa',  
    kind: 'full',  
    category: 'adhan',  
    names: { bg: 'Езан · Ал-Акса', en: 'Adhan · Al-Aqsa' },  
    file: 'adhan_aqsa',  
    durationSec: 225,  
  },  
  {  
    id: 'adhan_istanbul',  
    kind: 'full',  
    category: 'adhan',  
    names: { bg: 'Езан · Истанбул', en: 'Adhan · Istanbul' },  
    file: 'adhan_istanbul',  
    durationSec: 66,  
  },  
  // ---- кратки (iPhone и Android без точни аларми)  
  {  
    id: 'takbir_makkah',  
    kind: 'short',  
    category: 'takbir',  
    names: { bg: 'Текбири · Мека', en: 'Takbir · Makkah' },  
    file: 'takbir_makkah',  
    ext: 'wav',  
    durationSec: 30,  
  },  
  {  
    id: 'takbir_madinah',  
    kind: 'short',  
    category: 'takbir',  
    names: { bg: 'Текбири · Медина', en: 'Takbir · Madinah' },  
    file: 'takbir_madinah',  
    ext: 'wav',  
    durationSec: 25,  
  },  
  {  
    id: 'takbir_aqsa',  
    kind: 'short',  
    category: 'takbir',  
  },  
];
```

```

    names: { bg: 'Текбири · Ал-Акса', en: 'Takbir · Al-Aqsa' },
    file: 'takbir_aqsa',
    ext: 'wav',
    durationSec: 29,
  },
  {
    id: 'takbir_istanbul',
    kind: 'short',
    category: 'takbir',
    names: { bg: 'Текбири · Истанбул', en: 'Takbir · Istanbul' },
    file: 'takbir_istanbul',
    ext: 'wav',
    durationSec: 30,
  },
  {
    id: 'chime',
    kind: 'short',
    category: 'chime',
    names: { bg: 'Сигнал', en: 'Signal' },
    file: 'ezan_chime',
    ext: 'wav',
    durationSec: 2,
  },
  {
    id: 'chime_soft',
    kind: 'short',
    category: 'chime',
    names: { bg: 'Тих звън', en: 'Soft chime' },
    file: 'chime_soft',
    ext: 'wav',
    durationSec: 4,
  },
  {
    // звукът за известия, избран в настройките на телефона
    id: 'system',
    kind: 'short',
    category: 'chime',
    names: { bg: 'Звукът на телефона', en: "The phone's sound" },
    file: SYSTEM_SOUND,
    durationSec: 0,
  },
  // ---- още звуци за известията (assets/sounds/notify_*.wav, до 10 сек.)
  notifySound('airport', 'Летище', 'Airport', 3),
  notifySound('popup', 'Изскачане', 'Pop-up', 1),
  notifySound('right', 'Правилно', 'Right', 1),
  notifySound('modern', 'Модерен', 'Modern', 2),
  notifySound('magical', 'Вълшебен', 'Magical', 4),
  notifySound('chime_bright', 'Звънче', 'Chime', 3),
  notifySound('doorbell', 'Звънец на врата', 'Doorbell', 2),
  notifySound('celestial', 'Небесен', 'Celestial', 5),
  notifySound('bell_ding', 'Камбанка', 'Bell ding', 1),
  notifySound('crisp_ding', 'Чист звън', 'Crisp ding', 6),
  notifySound('shimmer', 'Блясък', 'Shimmer', 6),
  notifySound('smoke_alarm', 'Аларма за дим', 'Smoke alarm', 1),
  notifySound('swoop', 'Полъх', 'Swoop', 1),
  notifySound('retro', 'Ретро', 'Retro', 1),
  notifySound('robot', 'Робот', 'Robot', 2),
  notifySound('three_beep', 'Три сигнала', 'Three beeps', 1),
  notifySound('machine_beep', 'Машинен сигнал', 'Machine beep', 1),
  notifySound('water_drop', 'Капка', 'Water drop', 1),
  notifySound('hang_up', 'Затваряне', 'Hang up', 1),

```

```

    notifySound('standard', 'Стандартен', 'Standard', 5),
];

export const DEFAULT_FULL = 'adhan_makkah';
export const DEFAULT_SHORT = 'takbir_makkah';
/** Звукът на известията и напомнянията – по подразбиране звукът на телефона (решено 01.10.2026). */
export const DEFAULT_NOTIFY = 'system';

/** Молитвите, които имат аларма (изгревът е само известие). */
export const ALARM_PRAYERS: readonly PrayerId[] = ['fajr', 'dhuhr', 'asr', 'maghrib', 'isha'];

/** Свой звук, добавен от потребителя. */
export interface CustomSound {
    id: string;
    kind: SoundKind;
    name: string;
    /** Android: file:///... в паметта на приложението; iPhone: името на .caf файла в Library/Sounds. */
    file: string;
    /** Колко свири (на iPhone – след скъсяването до 30 сек.). */
    durationSec: number;
    /** iPhone: колко е бил записът, ако е скъсен. */
    originalSec?: number;
    /** 'notify' – свой звук за известия (до 10 сек.); иначе – за алармата. */
    use?: SoundUse;
}

export function customToDef(c: CustomSound): SoundDef {
    return {
        id: c.id,
        kind: c.kind,
        category: 'custom',
        names: { bg: c.name, en: c.name },
        file: c.file,
        ext: c.kind === 'short' ? 'caf' : undefined,
        durationSec: c.durationSec,
        originalSec: c.originalSec,
        custom: true,
        use: c.use ?? 'alarm',
    };
}

/** Всички звуци – вградените и своите. */
export function allSounds(custom: readonly CustomSound[]): SoundDef[] {
    return [...BUILTIN_SOUNDS, ...custom.map(customToDef)];
}

/** Звукът по id; ако го няма (изтрит свой звук) – резервният за вида. */
/**
 * За какво е звукът (решено 01.10.2026):
 * - 'alarm' – звукът на молитвата с камбанка „аларма“ (пълнен или кратък резервен): езан,
 *             текбири и своите звуци (мелодии – само свои, вградени няма);
 * - 'notify' – звукът на известията и напомнянията: само кратките сигнали (звън).
 */
export type SoundUse = 'alarm' | 'notify';

export function fitsUse(s: SoundDef, use: SoundUse): boolean {
    if (s.custom) return (s.use ?? 'alarm') === use;
    return use === 'notify' ? s.category === 'chime' : s.category !== 'chime';
}

/** Звукът по id; ако го няма (изтрит свой звук) или не е за това – резервният. */

```

```

export function findSound(
  id: string | undefined,
  kind: SoundKind,
  custom: readonly CustomSound[],
  use: SoundUse = 'alarm',
): SoundDef {
  const all = allSounds(custom);
  const found = all.find((s) => s.id === id && s.kind === kind && fitsUse(s, use));
  if (found) return found;
  const fallback = use === 'notify' ? DEFAULT_NOTIFY : kind === 'full' ? DEFAULT_FULL : DEFAULT_SHORT;
  return all.find((s) => s.id === fallback!);
}

/** Името на файла за известие (expo-notifications): „takbir_makkah.wav“ / „custom_17.caf“. */
export function notificationFileName(s: SoundDef): string {
  if (s.id === 'system') return 'default'; // expo-notifications: звукът по подразбиране на телефона
  if (s.custom) return s.file; // на iPhone: името в Library/Sounds (с разширението)
  return `${s.file}.${s.ext ?? 'wav'}`;
}

/** „3:42“ */
export function formatDuration(sec: number): string {
  const s = Math.max(0, Math.round(sec));
  return `${Math.floor(s / 60)}:${String(s % 60).padStart(2, '0')}`;
}

/** Проверка на своя звук преди добавяне. null – става; иначе причината. */
export function validateUpload(
  kind: SoundKind,
  durationSec: number,
  bytes: number | undefined,
  use: SoundUse = 'alarm',
): 'tooLong' | 'tooBig' | 'empty' | null {
  if (!(durationSec > 0.3)) return 'empty';
  if (bytes !== undefined && bytes > MAX_UPLOAD_BYTES) return 'tooBig';
  if (use === 'notify' && durationSec > NOTIFY_MAX_SEC + 0.5) return 'tooLong';
  if (kind === 'short' && durationSec > SHORT_MAX_SEC + 0.5) return 'tooLong';
  if (kind === 'full' && durationSec > FULL_MAX_SEC) return 'tooLong';
  return null;
}

/** Името от файла: „Ezan_Bania-Bashi.mp3“ → „Ezan Bania-Bashi“. */
export function nameFromFile(fileName: string): string {
  const base = fileName.replace(/\.([^.]+)$/, '').replace(/[_]+/g, ' ').trim();
  return base.length > 40 ? `${base.slice(0, 39)}...` : base || 'Sound';
}

```

sounds.ts src/store

- Пълният и краткият звук за всяка молитва, звукът на известията, своите звуци – запазват се в телефона.
- Махане на свой звук връща молитвите с него към звука по подразбиране.
- Миграция v1: запазеното „Сигнал“ от по-старата версия става „Звукът на телефона“ (еднократно).

src/store/sounds.ts

79 реда

```

import AsyncStorage from '@react-native-async-storage/async-storage';
import { create } from 'zustand';
import { createJSONStorage, persist } from 'zustand/middleware';

```

```

import type { PrayerId } from '@domain/prayers';
import {
  ALARM_PRAYERS,
  DEFAULT_FULL,
  DEFAULT_NOTIFY,
  DEFAULT_SHORT,
  type CustomSound,
  type SoundKind,
} from '@domain/sounds';

/** Избраните звуци за всяка молитва и своите звуци на потребителя. */
type PerPrayer = Partial<Record<PrayerId, string>>;

interface SoundsState {
  /** Пълният звук на алармата (Android). */
  full: PerPrayer;
  /** Краткият звук (iPhone; Android без точни аларми). */
  short: PerPrayer;
  /** Звукът на известията и напомнянията. */
  notify: string;
  custom: CustomSound[];

  setSound: (kind: SoundKind, prayer: PrayerId, id: string) => void;
  /** Звуките на тази молитва – за всички молитви. */
  applyToAll: (prayer: PrayerId) => void;
  setNotify: (id: string) => void;
  addCustom: (s: CustomSound) => void;
  /** Маха своя звук; молитвите с него се връщат към звука по подразбиране. */
  removeCustom: (id: string) => void;
}

const all = (id: string): PerPrayer => Object.fromEntries(ALARM_PRAYERS.map((p) => [p, id]));

export const useSounds = create<SoundsState>()(
  persist(
    (set) => ({
      full: all(DEFAULT_FULL),
      short: all(DEFAULT_SHORT),
      notify: DEFAULT_NOTIFY,
      custom: [],

      setSound: (kind, prayer, id) => set((s) => ({ [kind]: { ...s[kind], [prayer]: id } }) as
Partial<SoundsState>),
      applyToAll: (prayer) =>
        set((s) => ({
          full: all(s.full[prayer] ?? DEFAULT_FULL),
          short: all(s.short[prayer] ?? DEFAULT_SHORT),
        })),
      setNotify: (notify) => set({ notify }),
      addCustom: (c) => set((s) => ({ custom: [...s.custom, c] })),
      removeCustom: (id) =>
        set((s) => {
          const reset = (m: PerPrayer, def: string): PerPrayer =>
            Object.fromEntries(Object.entries(m).map(([k, v]) => [k, v === id ? def : v]));
          return {
            custom: s.custom.filter((c) => c.id !== id),
            full: reset(s.full, DEFAULT_FULL),
            short: reset(s.short, DEFAULT_SHORT),
            notify: s.notify === id ? DEFAULT_NOTIFY : s.notify,
          };
        });
    })
  )
);

```

```

    }},
  },
  {
    name: 'ezan.sounds',
    version: 1,
    storage: createJSONStorage(() => AsyncStorage),
    // v1 (01.10.2026): звукът на известията по подразбиране е звукът на телефона, а не „Сигнал“
    migrate: (persisted, version) => {
      const st = persisted as Partial<SoundsState>;
      if (version < 1 && st.notify === 'chime') st.notify = DEFAULT_NOTIFY;
      return st as SoundsState;
    },
    partialize: ({ full, short, notify, custom }) => ({ full, short, notify, custom }),
  },
),
);

```

Стъпка 5

Свои звуци и преслушване

Изборът на файл е еднакъв; обработката е различна за двете системи. Докато се реже, над екрана стои „Изрязване на откъса...“.

sounds.ts src/services

- **Android, пълен:** дължината (native), проверка, копие в `files/sounds/`.
- **Android, кратък:** Kotlin реже откъса и го записва в `Notifications/Ezan` (Android 10+, без разрешения) – оттам го чете системата за известия.
- **iPhone:** Swift преобразува файла в `.caf` в `Library/Sounds` и го скъсява до 30 сек.
- **Звук на известията:** първо се проверява дължината – над 10 сек. не се добавя (съобщение).
- Преслушване: на Android – native (MediaPlayer), на iPhone – expo-audio. Втори тап – спира.

src/services/sounds.ts

226 реда

```

import { createAudioPlayer, setAudioModeAsync, type AudioPlayer } from 'expo-audio';
import * as DocumentPicker from 'expo-document-picker';
import { Directory, File, Paths } from 'expo-file-system';
import { Platform } from 'react-native';
import { create } from 'zustand';

import {
  deleteNotificationSound,
  deleteShortSound,
  getAudioDuration,
  iosSoundsDirectoryUri,
  prepareNotificationSound,
  prepareShortSound,
  previewNativeSound,
  stopNativePreview,
} from '../modules/adhan-native';

import {
  FULL_MAX_SEC,
  NOTIFY_MAX_SEC,
  nameFromFile,
  notificationFileName,

```

```

SHORT_MAX_SEC,
validateUpload,
type CustomSound,
type SoundDef,
type SoundKind,
type SoundUse,
} from '@domain/sounds';
import { useSounds } from '@store/sounds';

/**
 * Звучите (етап 6): преслушване и своите звуци на потребителя.
 * - Android: преслушването и алармата са в native частта (MediaPlayer) – и за вградените
 *   звуци в res/raw, и за своите файлове в паметта на приложението.
 * - iPhone: преслушване с expo-audio; своите звуци се преобразуват в .caf в Library/Sounds,
 *   откъдето iOS ги пуска като звук на извешение. По-дългите от 30 сек. се скъсяват
 *   на естествена пауза, с плавно заглъхване (iOS не пуска звук над 30 сек.).
 */

/* ----- преслушване */

export const usePreview = create<{ playing: string | null }>(() => ({ playing: null }));

let iosPlayer: AudioPlayer | null = null;
let endTimer: ReturnType<typeof setTimeout> | undefined;

function sourceForIos(s: SoundDef): string | null {
  if (s.id === 'system') return null; // звукът на телефона не може да се пусне от приложението на iPhone
  if (s.custom) {
    const dir = iosSoundsDirectoryUri();
    return dir ? `${dir}${s.file}` : null;
  }
  // вградените кратки звуци са в приложението (expo-notifications ги слага там)
  return `${Paths.bundle.uri}${notificationFileName(s)}`;
}

export function stopPreview(): void {
  clearTimeout(endTimer);
  stopNativePreview();
  if (iosPlayer) {
    try {
      iosPlayer.pause();
      iosPlayer.remove();
    } catch {
      // нищо
    }
    iosPlayer = null;
  }
  usePreview.setState({ playing: null });
}

/** Пуска или спира звука (втори тап по същия – спира). */
export async function togglePreview(s: SoundDef): Promise<void> {
  const wasPlaying = usePreview.getState().playing === s.id;
  stopPreview();
  if (wasPlaying) return;

  let started = false;
  if (Platform.OS === 'android') {
    started = previewNativeSound(s.file);
  } else if (Platform.OS === 'ios') {
    const uri = sourceForIos(s);

```

```

    if (uri) {
      await setAudioModeAsync({ playsInSilentMode: true }).catch(() => {});
      iosPlayer = createAudioPlayer(uri);
      iosPlayer.addListener('playbackStatusUpdate', (st) => {
        if (st.didJustFinish) stopPreview();
      });
      iosPlayer.play();
      started = true;
    }
  }
  if (!started) return;
  usePreview.setState({ playing: s.id });
  // край на звука (native преслушването не съобщава кога свършва)
  endTimer = setTimeout(stopPreview, (s.id === 'system' ? 3 : Math.min(s.durationSec, FULL_MAX_SEC)) * 1000
+ 500);
}

/* ----- СВОИ ЗВУЦИ ----- */

export type AddResult =
  | { ok: true; sound: CustomSound }
  | { ok: false; reason: 'canceled' | 'tooLong' | 'tooBig' | 'unreadable' | 'unsupported'; name?: string;
durationSec?: number };

const PICK_TYPES = ['audio/*', 'video/mp4', 'application/ogg'];

function customDir(): Directory {
  const dir = new Directory(Paths.document, 'sounds');
  dir.create({ idempotent: true, intermediates: true });
  return dir;
}

/**
 * Избор на файл от телефона и добавяне като свой звук.
 * Android - пълен звук (за алармата); iPhone - кратък, до 30 сек. (за известието).
 */
export async function addCustomSound(
  kind: SoundKind,
  /** Файлът е избран - започва обработката (за индикатора „Изрязване...“). */
  onPicked?: (name: string) => void,
  /** 'notify' - звук за известия: до 10 сек., по-дълъг не се добавя. */
  use: SoundUse = 'alarm',
): Promise<AddResult> {
  if (Platform.OS === 'web') return { ok: false, reason: 'unsupported' };
  const picked = await DocumentPicker.getDocumentAsync({ type: PICK_TYPES, copyToCacheDirectory: true });
  if (picked.canceled || !picked.assets?.length) return { ok: false, reason: 'canceled' };
  const asset = picked.assets[0];
  const name = nameFromFile(asset.name);
  const id = `custom_${Date.now()}`;
  onPicked?.(name);
  const maxSec = use === 'notify' ? NOTIFY_MAX_SEC : SHORT_MAX_SEC;

  try {
    if (Platform.OS === 'ios') {
      // iPhone: преобразуване в .caf (iOS не приема mp3 за известия); по-дълъг - не се записва
      const file = `${id}.caf`;
      if (asset.size !== undefined && validateUpload('short', 1, asset.size) === 'tooBig') {
        return { ok: false, reason: 'tooBig', name };
      }
    }
    // звук за известия - до 10 сек., по-дълъг не се добавя
    if (use === 'notify') {

```

```

        const d = (await getAudioDuration(asset.uri)) ?? -1;
        const problem = validateUpload('short', d, asset.size, 'notify');
        if (problem) return { ok: false, reason: problem === 'empty' ? 'unreadable' : problem, name,
durationSec: d };
    }
    // дълъг запис се скъсява до 30 сек. - на пауза, с плавно заглъхване (виж AdhanNativeModule.swift)
    const r = await prepareNotificationSound(asset.uri, file, maxSec);
    if (!r) return { ok: false, reason: 'unsupported' };
    if (!r.ok || validateUpload('short', r.duration, asset.size)) {
        deleteNotificationSound(file);
        return { ok: false, reason: 'unreadable', name };
    }
    const sound: CustomSound = {
        id,
        kind: 'short',
        name,
        file,
        durationSec: r.duration,
        originalSec: r.trimmed ? r.originalDuration : undefined,
        use,
    };
    useSounds.getState().addCustom(sound);
    return { ok: true, sound };
}

// Android, кратък: откъс до 30 сек. (срез на пауза, заглъхване) в Notifications/Ezan -
// оттам го чете системата за известия (виж ShortSound.kt)
if (kind === 'short') {
    if (asset.size !== undefined && validateUpload('short', 1, asset.size) === 'tooBig') {
        return { ok: false, reason: 'tooBig', name };
    }
    if (use === 'notify') {
        const d = (await getAudioDuration(asset.uri)) ?? -1;
        const problem = validateUpload('short', d, asset.size, 'notify');
        if (problem) return { ok: false, reason: problem === 'empty' ? 'unreadable' : problem, name,
durationSec: d };
    }
    const r = await prepareShortSound(asset.uri, name, maxSec);
    if (!r) return { ok: false, reason: 'unsupported' };
    if (validateUpload('short', r.duration, asset.size)) {
        deleteShortSound(r.uri);
        return { ok: false, reason: 'unreadable', name };
    }
    const sound: CustomSound = {
        id,
        kind: 'short',
        name,
        file: r.uri,
        durationSec: r.duration,
        originalSec: r.trimmed ? r.originalDuration : undefined,
        use,
    };
    useSounds.getState().addCustom(sound);
    return { ok: true, sound };
}

// Android, пълен: копие в паметта на приложението; алармата го пуска оттам
const duration = (await getAudioDuration(asset.uri)) ?? -1;
const problem = validateUpload(kind, duration, asset.size);
if (problem) return { ok: false, reason: problem === 'empty' ? 'unreadable' : problem, name,
durationSec: duration };

```

```

const ext = (asset.name.match(/\.[a-z0-9]{2,4}$/i)?.[1] ?? 'mp3').toLowerCase();
const dest = new File(customDir(), `${id}.${ext}`);
await new File(asset.uri).copy(dest);
const sound: CustomSound = { id, kind, name, file: dest.uri, durationSec: duration };
useSounds.getState().addCustom(sound);
return { ok: true, sound };
} catch {
return { ok: false, reason: 'unreadable', name };
}
}

/** Маха своя звук (и файла му). Молитвите с него се връщат към звука по подразбиране. */
export function removeCustomSound(sound: CustomSound): void {
if (usePreview.getState().playing === sound.id) stopPreview();
try {
if (Platform.OS === 'ios') deleteNotificationSound(sound.file);
else if (sound.kind === 'short') deleteShortSound(sound.file);
else {
const f = new File(sound.file);
if (f.exists) f.delete();
}
} catch {
// файлът вече го няма
}
useSounds.getState().removeCustom(sound.id);
}
}

```

SoundSource.kt modules/adhan-native/android/src/main/java/expo/modules/adhannative

- Откъде идва звукът: `res/raw`, свой файл, `content://` или „system“. Ако липсва – резервният езан, накрая сигналът на телефона.

modules/adhan-native/android/src/main/java/expo/modules/adhannative/SoundSource.kt

123 реда

```
package expo.modules.adhannative
```

```

import android.content.Context
import android.media.AudioAttributes
import android.media.MediaMetadataRetriever
import android.media.MediaPlayer
import android.media.RingtoneManager
import android.net.Uri
import android.util.Log
import java.io.File

```

```
/**
```

```
 * Откъде да се вземе звукът (етап 6):
```

```
 * - вграден – името в res/raw без разширение („adhan_makkah“);
```

```
 * - свой – файл в паметта на приложението („file:///data/.../sounds/custom_17.mp3“).
```

```
 * Ако своят файл липсва (изтрит), се ползва вграденият резервен, а накрая – сигналът
```

```
 * за аларма на телефона.
```

```
*/
```

```
object SoundSource {
```

```
    private const val TAG = "AdhanSound"
```

```
    const val FALLBACK = "adhan_makkah"
```

```
    private fun isFile(source: String) = source.startsWith("file://") || source.startsWith("/")
```

```

    private fun fileOf(source: String): File = File(if (source.startsWith("file://")) Uri.parse(source).path
?: "" else source)

```

```

/** Задава източника на плейъра. true - вграден или свой звук; false - сигнала на телефона. */
fun setDataSource(context: Context, player: MediaPlayer, source: String): Boolean {
    if (source == "system") {
        // звукът за известия, избран в настройките на телефона
        player.setDataSource(context, android.provider.Settings.System.DEFAULT_NOTIFICATION_URI)
        return true
    }
    if (source.startsWith("content://")) {
        return try {
            player.setDataSource(context, Uri.parse(source))
            true
        } catch (e: Exception) {
            Log.w(TAG, "missing $source")
            setDataSource(context, player, FALLBACK)
        }
    }
    if (isFile(source)) {
        val f = fileOf(source)
        if (f.exists() && f.length() > 0) {
            player.setDataSource(f.absolutePath)
            return true
        }
        Log.w(TAG, "missing $source")
        return setDataSource(context, player, FALLBACK)
    }
    val resId = context.resources.getIdentifier(source, "raw", context.packageName)
    if (resId != 0) {
        val afd = context.resources.openRawResourceFd(resId)
        player.setDataSource(afd.fileDescriptor, afd.startOffset, afd.length)
        afd.close()
        return true
    }
    if (source != FALLBACK) return setDataSource(context, player, FALLBACK)
    player.setDataSource(context, RingtoneManager.getDefaultUri(RingtoneManager.TYPE_ALARM))
    return false
}

/** Дължината на звуков файл в секунди (mp3, m4a, mp4, wav, ogg, flac...). -1 - не става. */
fun durationSec(context: Context, uri: String): Double {
    val r = MediaMetadataRetriever()
    return try {
        if (isFile(uri)) r.setDataSource(fileOf(uri).absolutePath) else r.setDataSource(context,
Uri.parse(uri))
        val ms = r.extractMetadata(MediaMetadataRetriever.METADATA_KEY_DURATION)?.toLongOrNull() ?: -1L
        val hasAudio = r.extractMetadata(MediaMetadataRetriever.METADATA_KEY_HAS_AUDIO)
        if (hasAudio != null && hasAudio != "yes") -1.0 else if (ms > 0) ms / 1000.0 else -1.0
    } catch (e: Exception) {
        Log.w(TAG, "duration: ${e.message}")
        -1.0
    } finally {
        try {
            r.release()
        } catch (e: Exception) {
            // нищо
        }
    }
}

/** Преслушване в настройките - през потока за медия (силата на звука за музика). */

```

```

object SoundPreview {
    private var player: MediaPlayer? = null

    @Synchronized
    fun play(context: Context, source: String) {
        stop()
        val p = MediaPlayer()
        try {
            p.setAudioAttributes(
                AudioAttributes.Builder()
                    .setUsage(AudioAttributes.USAGE_MEDIA)
                    .setContentType(AudioAttributes.CONTENT_TYPE_MUSIC)
                    .build(),
            )
            SoundSource.setDataSource(context, p, source)
            p.setOnCompletionListener { stop() }
            p.prepare()
            p.start()
            player = p
        } catch (e: Exception) {
            Log.w("AdhanSound", "preview: ${e.message}")
            p.release()
        }
    }

    @Synchronized
    fun stop() {
        player?.let {
            try {
                it.stop()
            } catch (e: Exception) {
                // нищо
            }
            it.release()
        }
        player = null
    }
}

```

ShortSound.kt modules/adhan-native/android/src/main/java/expo/modules/adhannative

- MediaCodec декодира файла → тишината в началото се маха → срез на пауза и заглъхване → WAV в MediaStore (Notifications/Ezan).
- `createChannel` – канал със свой звук или със звука на телефона (expo-notifications приема само res/raw).

modules/adhan-native/android/src/main/java/expo/modules/adhannative/ShortSound.kt

318 реда

```

package expo.modules.adhannative

import android.app.NotificationChannel
import android.app.NotificationManager
import android.content.ContentValues
import android.content.Context
import android.media.AudioAttributes
import android.media.MediaCodec
import android.media.MediaExtractor
import android.media.MediaFormat
import android.net.Uri

```

```

import android.os.Build
import android.os.Environment
import android.provider.MediaStore
import java.io.File
import java.io.OutputStream
import java.nio.ByteBuffer
import java.nio.ByteOrder
import kotlin.math.abs
import kotlin.math.sqrt

/**
 * Свой кратък звук на Android (етап 6): откъс до 30 сек. от каквото избере потребителят.
 *
 * Краткият звук свири като звук на известие (когато „Alarms & reminders“ е изключено),
 * а системата за известия не чете файлове от паметта на приложението. Затова откъсът
 * се записва в общата папка Notifications/Ezan (Android 10+, без разрешения) и каналът
 * на известието сочи към него.
 *
 * Правилото за срез е същото като при вградените езани и на iPhone: тишината в началото
 * се маха, срез на последната истинска пауза между 15-ата и 30-ата сек., плавно заглъхване.
 */
object ShortSound {
    /** Колко от записа се чете най-много (тишина/въведение в началото + 30 сек.). */
    private const val READ_EXTRA_SEC = 60.0

    class Result(val uri: String, val duration: Double, val originalDuration: Double, val trimmed: Boolean)

    fun prepare(context: Context, source: String, title: String, maxSec: Double): Result {
        val pcm = decode(context, Uri.parse(source), maxSec + READ_EXTRA_SEC)
        val rate = pcm.rate
        val start = leadingSilence(pcm.samples, pcm.count, rate)
        val samples = pcm.samples.copyOfRange(start, pcm.count)
        val (end, fade) = cutPoint(samples, rate, maxSec)
        fadeOut(samples, end, fade)
        val wav = wavBytes(samples, end, rate)

        val original = if (pcm.totalSec > 0) pcm.totalSec else pcm.count.toDouble() / rate
        val playable = original - start.toDouble() / rate
        val uri = save(context, wav, title)
        return Result(uri, end.toDouble() / rate, original, playable > maxSec + 0.25)
    }

    /** Маха откъса (content:// от MediaStore или файл). */
    fun delete(context: Context, uri: String) {
        try {
            if (uri.startsWith("content://")) {
                context.contentResolver.delete(Uri.parse(uri), null, null)
            } else {
                File(Uri.parse(uri).path ?: uri).delete()
            }
        } catch (e: Exception) {
            // вече го няма
        }
    }

    /**
     * Канал за известие със свой звук (expo-notifications приема само звуци от res/raw).
     * alarm - езанът като известие: потокът за аларми и най-висока важност.
     */
    fun createChannel(
        context: Context,

```

```

id: String,
name: String,
uri: String,
alarm: Boolean,
vibrate: Boolean,
pattern: LongArray?,
) {
    if (Build.VERSION.SDK_INT < Build.VERSION_CODES.O) return
    val nm = context.getSystemService(Context.NOTIFICATION_SERVICE) as NotificationManager
    val channel = NotificationChannel(
        id,
        name,
        if (alarm) NotificationManager.IMPORTANCE_HIGH else NotificationManager.IMPORTANCE_DEFAULT,
    )
    val attrs = AudioAttributes.Builder()
        .setUsage(if (alarm) AudioAttributes.USAGE_ALARM else AudioAttributes.USAGE_NOTIFICATION)
        .setContentType(AudioAttributes.CONTENT_TYPE_SONIFICATION)
        .build()
    // файл от паметта на приложението системата не може да чете - тогава звукът по подразбиране
    val sound = when {
        uri == "system" -> android.provider.Settings.System.DEFAULT_NOTIFICATION_URI
        uri.startsWith("content://") -> Uri.parse(uri)
        else -> null
    }
    channel.setSound(sound, attrs)
    channel.enableVibration(vibrate)
    if (vibrate && pattern != null) channel.vibrationPattern = pattern
    channel.lockscreenVisibility = android.app.Notification.VISIBILITY_PUBLIC
    nm.createNotificationChannel(channel)
}

/* ----- четене */

private class Pcm(val samples: ShortArray, val count: Int, val rate: Int, val totalSec: Double)

/** Всеки звук (mp3, m4a, mp4, wav, ogg, flac) → 16-битови семпли, моно, честотата на източника. */
private fun decode(context: Context, uri: Uri, maxSec: Double): Pcm {
    val extractor = MediaExtractor()
    try {
        extractor.setDataSource(context, uri, null)
        var track = -1
        var format: MediaFormat? = null
        for (i in 0 until extractor.trackCount) {
            val f = extractor.getTrackFormat(i)
            if ((f.getString(MediaFormat.KEY_MIME) ?: "").startsWith("audio/")) {
                track = i
                format = f
                break
            }
        }
        if (track < 0 || format == null) throw IllegalArgumentException("no audio")
        extractor.selectTrack(track)
        val totalSec =
            if (format.containsKey(MediaFormat.KEY_DURATION)) format.getLong(MediaFormat.KEY_DURATION) /
            1_000_000.0 else -1.0
        var rate = format.getInteger(MediaFormat.KEY_SAMPLE_RATE)
        var channels = format.getInteger(MediaFormat.KEY_CHANNEL_COUNT).coerceAtLeast(1)

        val codec = MediaCodec.createDecoderByType(format.getString(MediaFormat.KEY_MIME)!!)
        codec.configure(format, null, null, 0)
        codec.start()
    }
}

```

```

var out = ShortArray((rate * (maxSec + 1)).toInt().coerceAtLeast(1024))
var n = 0
val info = MediaCodec.BufferInfo()
var inputDone = false
var done = false
var idle = 0
try {
    while (!done && idle < 500) {
        if (!inputDone) {
            val ii = codec.dequeueInputBuffer(10_000)
            if (ii >= 0) {
                val buf = codec.getInputBuffer(ii)!!
                val size = extractor.readSampleData(buf, 0)
                if (size < 0) {
                    codec.queueInputBuffer(ii, 0, 0, 0, MediaCodec.BUFFER_FLAG_END_OF_STREAM)
                    inputDone = true
                } else {
                    codec.queueInputBuffer(ii, 0, size, extractor.sampleTime, 0)
                    extractor.advance()
                }
            }
        }
        val oi = codec.dequeueOutputBuffer(info, 10_000)
        if (oi == MediaCodec.INFO_OUTPUT_FORMAT_CHANGED) {
            val f = codec.outputFormat
            rate = f.getInteger(MediaFormat.KEY_SAMPLE_RATE)
            channels = f.getInteger(MediaFormat.KEY_CHANNEL_COUNT).coerceAtLeast(1)
        } else if (oi >= 0) {
            idle = 0
            val ob: ByteBuffer = codec.getOutputBuffer(oi)!!
            ob.position(info.offset)
            ob.limit(info.offset + info.size)
            val sb = ob.slice().order(ByteOrder.nativeOrder()).asShortBuffer()
            val frames = sb.remaining() / channels
            for (fr in 0 until frames) {
                var sum = 0
                for (c in 0 until channels) sum += sb.get().toInt()
                if (n >= out.size) out = out.copyOf(out.size * 2)
                out[n++] = (sum / channels).toShort()
            }
            codec.releaseOutputBuffer(oi, false)
            if ((info.flags and MediaCodec.BUFFER_FLAG_END_OF_STREAM) != 0 || n >= rate * maxSec) done =
true
        } else if (inputDone) {
            idle++
        }
    }
} finally {
    try {
        codec.stop()
    } catch (e: Exception) {
        // нищо
    }
    codec.release()
}
if (n == 0) throw IllegalArgumentException("no audio")
return Pcm(out, n, rate, totalSec)
} finally {
    extractor.release()
}
}

```

```

/* ----- срез */

/** Колко семпла тишина има в началото (под 3% от най-силното място), с 50 мс запас. */
private fun leadingSilence(s: ShortArray, count: Int, rate: Int): Int {
    var peak = 0
    for (i in 0 until count) {
        val a = abs(s[i].toInt())
        if (a > peak) peak = a
    }
    if (peak == 0) return 0
    val threshold = maxOf(peak * 0.03, 200.0)
    val win = (rate / 50).coerceAtLeast(1)
    var i = 0
    while (i + win <= count) {
        if (rms(s, i, win) > threshold) return maxOf(0, i - (0.05 * rate).toInt())
        i += win
    }
    return 0
}

private fun rms(s: ShortArray, start: Int, win: Int): Double {
    var sum = 0.0
    val end = minOf(start + win, s.size)
    for (k in start until end) {
        val v = s[k].toDouble()
        sum += v * v
    }
    return sqrt(sum / win)
}

/**
 * Докъде да свири и колко семпла да заглъхва. До maxSec – целият звук.
 * По-дълъг: последната истинска пауза (тихо поне 150 мс) между 15-ата секунда и края –
 * първо дълбоките (под 12% от средната сила), после по-меките (под 35%); срез в средата ѝ
 * с 0,5 сек. заглъхване. Без пауза – срез на maxSec с 3 сек. заглъхване.
 */
private fun cutPoint(s: ShortArray, rate: Int, maxSec: Double): Pair<Int, Int> {
    val maxN = (maxSec * rate).toInt()
    if (s.size <= maxN) return Pair(s.size, minOf(s.size, (0.01 * rate).toInt()))
    val win = (rate / 10).coerceAtLeast(2)
    val levels = ArrayList<Double>()
    var i = 0
    while (i + win <= maxN) {
        levels.add(rms(s, i, win))
        i += win / 2
    }
    levels.sort()
    val median = if (levels.isEmpty()) 0.0 else levels[levels.size / 2]
    if (median <= 0) return Pair(maxN, 3 * rate)
    val to = maxN - win - win / 2
    var deep = -1
    var soft = -1
    var j = 15 * rate
    while (j <= to) {
        val level = maxOf(rms(s, j, win), rms(s, j + win / 2, win))
        if (level < 0.12 * median) {
            deep = j
        } else if (level < 0.35 * median) {
            soft = j
        }
    }
}

```

```

        j += win / 2
    }
    val pick = if (deep >= 0) deep else soft
    if (pick >= 0) return Pair(pick + (3 * win) / 4, rate / 2)
    return Pair(maxN, 3 * rate)
}

private fun fadeOut(s: ShortArray, end: Int, length: Int) {
    val n = minOf(length, end)
    for (k in 0 until n) {
        val g = 1 - (k + 1).toDouble() / n
        val idx = end - n + k
        s[idx] = (s[idx] * g * g).toInt().toShort()
    }
}

/* ----- запис */

private fun wavBytes(s: ShortArray, count: Int, rate: Int): ByteArray {
    val data = count * 2
    val b = ByteBuffer.allocate(44 + data).order(ByteOrder.LITTLE_ENDIAN)
    b.put("RIFF".toByteArray()).putInt(36 + data).put("WAVE".toByteArray())
    b.put("fmt ".toByteArray()).putInt(16).putShort(1).putShort(1).putInt(rate).putInt(rate *
2).putShort(2).putShort(16)
    b.put("data".toByteArray()).putInt(data)
    for (i in 0 until count) b.putShort(s[i])
    return b.array()
}

/** Android 10+: Notifications/Ezan (системата го чете); по-старите - паметта на приложението. */
private fun save(context: Context, wav: ByteArray, title: String): String {
    val safe = title.replace(Regex("[\\\\\\/:?*\\\"<>|]"), " ").trim().take(60).ifEmpty { "Sound" }
    val fileName = "Ezan - $safe ${System.currentTimeMillis() % 100000}.wav"
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.Q) {
        val resolver = context.contentResolver
        val values = ContentValues().apply {
            put(MediaStore.Audio.Media.DISPLAY_NAME, fileName)
            put(MediaStore.Audio.Media.TITLE, "Ezan - $safe")
            put(MediaStore.Audio.Media.MIME_TYPE, "audio/x-wav")
            put(MediaStore.Audio.Media.RELATIVE_PATH, Environment.DIRECTORY_NOTIFICATIONS + "/Ezan")
            put(MediaStore.Audio.Media.IS_NOTIFICATION, 1)
            put(MediaStore.Audio.Media.IS_MUSIC, 0)
            put(MediaStore.Audio.Media.IS_PENDING, 1)
        }
        val collection = MediaStore.Audio.Media.getContentUri(MediaStore.VOLUME_EXTERNAL_PRIMARY)
        val uri = resolver.insert(collection, values) ?: throw IllegalStateException("insert")
        try {
            val os: OutputStream = resolver.openOutputStream(uri) ?: throw IllegalStateException("open")
            os.use { it.write(wav) }
            values.clear()
            values.put(MediaStore.Audio.Media.IS_PENDING, 0)
            resolver.update(uri, values, null, null)
        } catch (e: Exception) {
            resolver.delete(uri, null, null)
            throw e
        }
        return uri.toString()
    }
}

// Android 7-9: там „Alarms & reminders“ не съществува - винаги свири пълната аларма,
// откъсът е само за преслушване
val dir = File(context.filesDir, "sounds").apply { mkdirs() }

```

```

    val f = File(dir, "short_${System.currentTimeMillis()}.wav")
    f.writeBytes(wav)
    return Uri.fromFile(f).toString()
}
}

```

AdhanNativeModule.swift modules/adhan-native/ios

- AVAssetReader → тишината в началото се маха → **cutPoint** намира паузата → **fadeOut** → AVAudioFile записва CAF.

modules/adhan-native/ios/AdhanNativeModule.swift

250 реда

```

import AVFoundation
import ExpoModulesCore

/**
 * Native частта на Езан за iPhone (етап 6): своите звуци на потребителя.
 *
 * iOS приема за звук на известие само файлове до 30 сек. във формат aiff, wav или caf
 * (некомпресиран звук), записани в Library/Sounds на приложението. Затова избраният
 * mp3 / m4a / mp4 / wav се преобразува тук в .caf (16 бита, моно, 22 050 Hz).
 */
public class AdhanNativeModule: Module {
    public func definition() -> ModuleDefinition {
        Name("AdhanNative")

        // Дължината на звуков файл в секунди; -1 - не е звук или не може да се прочете.
        AsyncFunction("getAudioDuration") { (uri: String) async -> Double in
            let asset = AVURLAsset(url: Self.fileURL(uri))
            guard let tracks = try? await asset.loadTracks(withMediaType: .audio), !tracks.isEmpty,
                  let duration = try? await asset.load(.duration) else {
                return -1
            }
            let seconds = CMTimeGetSeconds(duration)
            return seconds.isFinite ? seconds : -1
        }

        // Преобразува файла в Library/Sounds/<name> (.caf, 16 бита, моно, 22 050 Hz).
        // Ако е по-дълъг от maxSec, го скъсява: реже на тиха пауза (мюзинът си поема дъх)
        // между 15-ата секунда и края, с плавно заглъхване; без пауза - на maxSec, с по-дълго заглъхване.
        // Връща { duration (на скъсения), originalDuration, trimmed, ok }.
        AsyncFunction("prepareNotificationSound") { (uri: String, name: String, maxSec: Double) async throws ->
[String: Any] in
            let asset = AVURLAsset(url: Self.fileURL(uri))
            let tracks = try await asset.loadTracks(withMediaType: .audio)
            guard let track = tracks.first else {
                throw NoAudioException()
            }
            let original = CMTimeGetSeconds(try await asset.load(.duration))
            guard original.isFinite, original > 0.3 else {
                throw NoAudioException()
            }
            // с запас: много записи започват с тишина или въведение (при Мишари - 19 сек. тишина)
            var samples = try Self.readPCM(asset: asset, track: track, seconds: min(original, maxSec + 60))
            let start = Self.leadingSilence(samples)
            if start > 0 {
                samples.removeFirst(start)
            }
            let playable = original - Double(start) / Self.rate

```

```

        let (end, fade) = Self.cutPoint(samples, maxSec: maxSec)
        Self.fadeOut(&samples, end: end, length: fade)
        let dest = try Self.soundsDirectory().appendingPathComponent(name)
        try? FileManager.default.removeItem(at: dest)
        try Self.writeCAF(samples, count: end, to: dest)
        return [
            "duration": Double(end) / Self.rate,
            "originalDuration": original,
            "trimmed": playable > maxSec + 0.25,
            "ok": true,
        ]
    }

    Function("deleteNotificationSound") { (name: String) in
        if let dir = try? Self.soundsDirectory() {
            try? FileManager.default.removeItem(at: dir.appendingPathComponent(name))
        }
    }

    // Пътят до Library/Sounds (file:///...) - за преслушване на своите звуци.
    Function("soundsDirectoryUri") { () -> String in
        (try? Self.soundsDirectory().absoluteString) ?? ""
    }
}

private static func fileURL(_ uri: String) -> URL {
    if uri.hasPrefix("file:///"), let url = URL(string: uri) {
        return url
    }
    return URL(fileURLWithPath: uri)
}

private static func soundsDirectory() throws -> URL {
    let library = try FileManager.default.url(for: .libraryDirectory, in: .userDomainMask, appropriateFor:
nil, create: true)
    let dir = library.appendingPathComponent("Sounds", isDirectory: true)
    try FileManager.default.createDirectory(at: dir, withIntermediateDirectories: true)
    return dir
}

private static let rate = 22050.0

/** Всеки звук (mp3, m4a, mp4, wav...) → 16-битови семпли, моно, 22 050 Hz - първите `seconds`. */
private static func readPCM(asset: AVURLAsset, track: AVAssetTrack, seconds: Double) throws -> [Int16] {
    let pcm: [String: Any] = [
        AVFormatIDKey: kAudioFormatLinearPCM,
        AVSampleRateKey: rate,
        AVNumberOfChannelsKey: 1,
        AVLinearPCMBitDepthKey: 16,
        AVLinearPCMIsFloatKey: false,
        AVLinearPCMIsBigEndianKey: false,
        AVLinearPCMIsNonInterleaved: false,
    ]
    let reader = try AVAssetReader(asset: asset)
    reader.timeRange = CMTimeRange(start: .zero, duration: CMTime(seconds: seconds, preferredTimescale:
600))
    let output = AVAssetReaderTrackOutput(track: track, outputSettings: pcm)
    // add() хвърля Objective-C изключение, ако не може - проверка преди това
    guard reader.canAdd(output) else {
        throw ConversionException()
    }
}

```

```

reader.add(output)
guard reader.startReading() else {
    throw ConversionException()
}
var samples = [Int16]()
samples.reserveCapacity(Int(rate * (seconds + 1)))
while let buffer = output.copyNextSampleBuffer() {
    guard let block = CMSampleBufferGetDataBuffer(buffer) else { continue }
    let length = CMBlockBufferGetDataLength(block)
    if length < 2 { continue }
    var chunk = [Int16](repeating: 0, count: length / 2)
    let status = chunk.withUnsafeMutableBytes { raw -> OSStatus in
        guard let base = raw.baseAddress else { return -1 }
        return CMBlockBufferCopyDataBytes(block, atOffset: 0, dataLength: raw.count, destination: base)
    }
    if status == noErr {
        samples.append(contentsOf: chunk)
    }
}
if reader.status == .failed || samples.isEmpty {
    throw ConversionException()
}
return samples
}

/** Колко семпла тишина има в началото (под 3% от най-силното място), с 50 мс запас. */
private static func leadingSilence(_ s: [Int16]) -> Int {
    var peak = 0
    for v in s {
        let a = abs(Int(v))
        if a > peak { peak = a }
    }
    if peak == 0 { return 0 }
    let threshold = max(Double(peak) * 0.03, 200)
    let win = Int(rate / 50) // 20 мс
    var i = 0
    while i + win <= s.count {
        var sum = 0.0
        for k in i..

```

```

let win = Int(rate / 10)
let rms: (Int) -> Double = { start in
    var sum = 0.0
    for i in start..

```

```

        channel[0].update(from: base, count: count)
    }
}
do {
    let file = try AVAudioFile(forWriting: dest, settings: format.settings, commonFormat:
.pcmFormatInt16, interleaved: true)
    try file.write(from: buffer)
} catch {
    try? FileManager.default.removeItem(at: dest)
    throw ConversionException()
}
}
}

internal final class NoAudioException: Exception {
    override var reason: String {
        "The file has no audio"
    }
}

internal final class ConversionException: Exception {
    override var reason: String {
        "The sound could not be converted"
    }
}

```

AdhanNative.podspec modules/adhan-native/ios

- iOS частта на модула (iOS 16.4+).

modules/adhan-native/ios/AdhanNative.podspec

21 реда

```

Pod::Spec.new do |s|
    s.name           = 'AdhanNative'
    s.version        = '0.2.0'
    s.summary        = 'Native helpers for the Adhan app (sounds for notifications on iPhone)'
    s.description    = 'Converts the user own sounds to a format iOS accepts for notification sounds.'
    s.license        = 'MIT'
    s.author         = 'Ilko Adamov'
    s.homepage       = 'https://github.com/ilkoadamov'
    s.platforms      = { :ios => '16.4' }
    s.swift_version  = '5.9'
    s.source         = { git: '' }
    s.static_framework = true

    s.dependency 'ExpoModulesCore'

    s.source_files = "**/*.{h,m,swift}"
    s.pod_target_xcconfig = {
        'DEFINES_MODULE' => 'YES',
        'SWIFT_COMPILATION_MODE' => 'wholemodule'
    }
end

```

Стъпка 6

Известията и алармата

На Android звукът е на канала и не може да се смени – затова каналът е за всеки звук.

notifications.ts src/services

- `ensureChannel` – „ch4-n-...” (известие) и „ch4-a-...” (езанът като известие, през потока за аларми); с и без вибрация. Своите звуци и звукът на телефона – през Kotlin.
- `cleanupChannels` – трие каналите на сменените звуци и тези отпреди етап 6.
- Алармата получава избрания пълен звук и името му (под вълната на екрана „Аларма“); при свой звук бутонът е „Спри алармата“.
- Пробното известие е със звука на известията.

src/services/notifications.ts

415 реда

```
import * as Notifications from 'expo-notifications';
import { Linking, Platform } from 'react-native';
import { create } from 'zustand';

import {
  canScheduleExactAlarms,
  canUseFullScreenIntent,
  createSoundChannel,
  hasNativeAlarms,
  isIgnoringBatteryOptimizations,
  openExactAlarmSettings,
  setAlarms,
  type NativeAlarm,
} from '../modules/adhan-native';

import { planNotifications, type PlannedNotification } from '@domain/notifications';
import { PRAYERS, type PrayerId } from '@domain/prayers';
import { findSound, notificationFileName, type SoundDef } from '@domain/sounds';
import { getI18n } from '@i18n';
import { PHASE_GRADIENTS } from '@theme/gradients';
import { useAlertPrefs } from '@store/alertPrefs';
import { selectLocation, selectTimesOptions, useSettings } from '@store/settings';
import { useSounds } from '@store/sounds';

/**
 * Известията за намаз – само локални (без интернет, без Firebase/FCM и без HMS Push).
 * Android планира всяко известие с AlarmManager, iPhone – с UNUserNotificationCenter.
 * Затова работят еднакво на телефони с Google услуги, на Huawei без тях и на iPhone.
 */

export type NotificationPermission = 'granted' | 'denied' | 'undetermined' | 'unavailable';

interface NotificationStatus {
  /** false, докато разрешението още не е проверено (първите мигове след стартиране). */
  checked: boolean;
  permission: NotificationPermission;
  /** Android 13+/iOS: може ли приложението пак да покаже системния въпрос. */
  canAskAgain: boolean;
  /** Android 12+: точни известия („Аларми и напомняния“); null – неприложимо. */
  exact: boolean | null;
  /** Колко известия са планирани и до кога (заедно с алармите). */
  count: number;
  until: Date | null;
  /** Android 14+: „Аларма на цял екран“; null – неприложимо. */
  fullScreen: boolean | null;
  /** Android: true – без ограничения на батерията; null – неприложимо. */
  battery: boolean | null;
  /** Колко аларми с езан са планирани в native частта (Android). */
}
```

```

alarms: number;
}

export const useNotificationStatus = create<NotificationStatus>(() => ({
  checked: false,
  permission: Platform.OS === 'web' ? 'unavailable' : 'undetermined',
  canAskAgain: true,
  exact: null,
  count: 0,
  until: null,
  fullScreen: null,
  battery: null,
  alarms: 0,
}));

const SUPPORTED = Platform.OS === 'android' || Platform.OS === 'ios';

/* ----- звуци и канали (Android) */

/**
 * Етап 6: всяка молитва има свой звук. На Android звукът е на канала и не може да се
 * смени след създаването му – затова каналът е за двойката „вид + звук“ (+ с/без вибрация):
 * „ch4-n-chime_soft-v“ – известие/напомняне, „ch4-a-takbir_makka-v“ – езанът като известие
 * (без „Аларми и напомняния“), през потока за аларми. Ненужните канали се трият.
 */
type ChannelUse = 'notify' | 'alarm';
const CHANNEL_PREFIX = 'ch4-';
/** Каналите отпреди етап 6 („prayer-v3“, „adhan-s2“...). */
const OLD_CHANNEL = /^(prayer|adhan|reminder)-[vs]\d$/;

const channelId = (use: ChannelUse, sound: SoundDef, vibrate: boolean) =>
  `${CHANNEL_PREFIX}${use === 'alarm' ? 'a' : 'n'}-${(sound.custom ? sound.id : sound.file).replace(/[^a-z0-9_]/gi, '_')}-${vibrate ? 'v' : 's'}`;

/** Известие и напомняне: едно дълго вибриране (1 сек.). */
const VIBRATION_ONCE = [0, 1000];
/** Езанът като известие (до 30 сек.): 1 сек. вибрация / 0,7 сек. пауза. */
const VIBRATION_ALARM = [0, ...Array.from({ length: 16 }, () => [1000, 700]).flat()];
const vibrationFor = (use: ChannelUse) => (use === 'alarm' ? VIBRATION_ALARM : VIBRATION_ONCE);

/** Каналите, създадени в тази сесия (създаването е бавно – не всеки път). */
const created = new Set<string>();

async function ensureChannel(use: ChannelUse, sound: SoundDef, vibrate: boolean): Promise<string> {
  const id = channelId(use, sound, vibrate);
  if (Platform.OS !== 'android' || created.has(id)) return id;
  const { lang, t } = getI18n();
  const name = `${use === 'alarm' ? t.notifications.channelAdhan : t.notifications.channelPrayer} ·
  ${sound.names[lang]}`;
  // свой кратък звук (откъсът в Notifications/Ezan) и звукът на телефона – каналът се създава
  // в native частта, защото expo-notifications приема само звуци от res/raw
  if (sound.custom || sound.id === 'system') {
    createSoundChannel(id, name, sound.file, use === 'alarm', vibrate, vibrationFor(use));
    created.add(id);
    return id;
  }
  const { AndroidAudioUsage: Usage, AndroidAudioContentType: Content, AndroidImportance: Imp } =
    Notifications;
  await Notifications.setNotificationChannelAsync(id, {
    name,
    importance: use === 'alarm' ? Imp.MAX : Imp.HIGH,
  });
}

```

```

    sound: notificationFileName(sound),
    // езанът - през потока за аларми: не зависи от силата на звука за известия и от безшумния режим
    audioAttributes: { usage: use === 'alarm' ? Usage.ALARM : Usage.NOTIFICATION, contentType:
Content.SONIFICATION },
    enableVibrate: vibrate,
    vibrationPattern: vibrate ? vibrationFor(use) : null,
    lockscreenVisibility: Notifications.AndroidNotificationVisibility.PUBLIC,
  });
  created.add(id);
  return id;
}

/** Три канала, които вече не трябва (стари версии, сменени звуци, вибрация). */
async function cleanupChannels(wanted: Set<string>): Promise<void> {
  if (Platform.OS !== 'android') return;
  const channels = await Notifications.getNotificationChannelsAsync();
  for (const c of channels) {
    if ((c.id.startsWith(CHANNEL_PREFIX) && !wanted.has(c.id)) || OLD_CHANNEL.test(c.id)) {
      await Notifications.deleteNotificationChannelAsync(c.id);
      created.delete(c.id);
    }
  }
}

/** Избраните звуци - от настройките (src/store/sounds.ts). */
function soundChoice() {
  const s = useSounds.getState();
  return {
    notify: findSound(s.notify, 'short', s.custom, 'notify'),
    short: (p: PrayerId) => findSound(s.short[p], 'short', s.custom),
    full: (p: PrayerId) => findSound(s.full[p], 'full', s.custom),
  };
}

/* ----- разрешение */

function toPermission(p: Notifications.NotificationPermissionsStatus): NotificationPermission {
  if (Platform.OS === 'ios') {
    const s = p.ios?.status;
    if (
      s === Notifications.IosAuthorizationStatus.AUTHORIZED ||
      s === Notifications.IosAuthorizationStatus.PROVISIONAL ||
      s === Notifications.IosAuthorizationStatus.EPHEMERAL
    ) {
      return 'granted';
    }
    return s === Notifications.IosAuthorizationStatus.DENIED ? 'denied' : 'undetermined';
  }
  if (p.granted) return 'granted';
  return p.status === 'undetermined' ? 'undetermined' : 'denied';
}

export async function refreshPermission(): Promise<NotificationPermission> {
  if (!SUPPORTED) return 'unavailable';
  const p = await Notifications.getPermissionsAsync();
  const permission = toPermission(p);
  useNotificationStatus.setState({
    checked: true,
    permission,
    canAskAgain: p.canAskAgain,
    exact: canScheduleExactAlarms(),
  });
}

```

```

    fullScreen: canUseFullScreenIntent(),
    battery: isIgnoringBatteryOptimizations(),
  });
  return permission;
}

/**
 * Пита за разрешение. Ако системата вече не показва въпроса (отказано завинаги) –
 * отваря настройките на приложението в телефона.
 */
export async function requestPermission(): Promise<NotificationPermission> {
  if (!SUPPORTED) return 'unavailable';
  const current = await Notifications.getPermissionsAsync();
  if (toPermission(current) === 'denied' && !current.canAskAgain) {
    await Linking.openSettings();
    return 'denied';
  }
  const p = await Notifications.requestPermissionsAsync({
    ios: { allowAlert: true, allowSound: true, allowBadge: false },
  });
  const permission = toPermission(p);
  useNotificationStatus.setState({ permission, canAskAgain: p.canAskAgain });
  if (permission === 'granted') rescheduleNotifications();
  return permission;
}

export { openExactAlarmSettings };

/* ----- планиране */

/** Всичко, което определя едно известие. Ако се смени, известието се пренасрочва. */
function signatureOf(n: PlannedNotification, channel: string, exact: boolean | null): string {
  return `${n.at.getTime()}|${n.sound}|${channel}|${exact}|${n.title}|${n.body}`;
}

let running: Promise<void> | null = null;
let again = false;

/**
 * Привежда планираните известия в съответствие с настройките.
 * Вика се при стартиране, при връщане в приложението, при всяка промяна на мястото,
 * часовете, камбанките, напомнянето и вибрацията, и от фоновата задача.
 * Паралелните извиквания се обединяват.
 */
export function rescheduleNotifications(): Promise<void> {
  if (running) {
    again = true;
    return running;
  }
  running = doReschedule()
    .catch((e) => console.warn('[notifications]', e))
    .finally(() => {
      running = null;
      if (again) {
        again = false;
        rescheduleNotifications();
      }
    });
  return running;
}

```

```

async function doReschedule(): Promise<void> {
  if (!SUPPORTED) return;
  const permission = await refreshPermission();
  const exact = useNotificationStatus.getState().exact;

  const s = useSettings.getState();
  const { lang, t } = getI18n();
  const plan =
    permission === 'granted'
      ? planNotifications({
          now: new Date(),
          options: selectTimesOptions(s).options,
          modes: useAlertPrefs.getState().modes,
          reminderMinutes: s.reminderMinutes,
          placeName: selectLocation(s).names[lang],
          texts: { prayers: t.prayers, ...t.notifications },
        })
      : [];

  const sounds = soundChoice();

  // Android (етап 5): езанът е истинска аларма в native частта - пълен звук, „Спри“,
  // екран „Аларма“. Без точни аларми (setAlarmClock не става) - остава звукът на известието.
  let alarmCount = 0;
  let notifications = plan;
  if (nativeAlarmsEnabled(exact)) {
    const alarms = plan.filter(isAlarm);
    notifications = plan.filter((n) => !isAlarm(n));
    alarmCount = Math.max(
      0,
      setAlarms(alarms.map((n) => toNativeAlarm(n, s.vibrate, selectLocation(s).names[lang],
sounds.full(n.prayer!)))),
    );
  } else if (hasNativeAlarms()) {
    setAlarms([]);
  }

  // Сравнява с вече планираните: трие само променените и липсващите, добавя само новите.
  const existing = await Notifications.getAllScheduledNotificationsAsync();
  const ours = new Map(
    existing
      .filter((r) => r.identifier.startsWith('ezan-'))
      .map((r) => [r.identifier, (r.content.data as { sig?: string } | null)?.sig ?? '']),
  );

  // кой звук за всяко известие: езанът като известие - краткият звук на молитвата; другите - звукът на известията
  const wanted = [];
  for (const n of notifications) {
    const use: ChannelUse = isAlarm(n) ? 'alarm' : 'notify';
    const sound = use === 'alarm' ? sounds.short(n.prayer!) : sounds.notify;
    const channel = await ensureChannel(use, sound, s.vibrate);
    wanted.push({ n, use, sound, channel, sig: signatureOf(n, channel, exact) + '|' +
notificationFileName(sound) });
  }
  const wantedIds = new Set(wanted.map((w) => w.n.id));

  for (const [id, sig] of ours) {
    const w = wanted.find((x) => x.n.id === id);
    if (!wantedIds.has(id) || !w || w.sig !== sig) {
      await Notifications.cancelScheduledNotificationAsync(id);
    }
  }
}

```

```

        ours.delete(id);
    }
}

for (const { n, use, sound, channel, sig } of wanted) {
    if (ours.has(n.id)) continue;
    await Notifications.scheduleNotificationAsync({
        identifier: n.id,
        content: {
            title: n.title,
            body: n.body,
            // на Android звукът е от канала, но без име на звук известието би било „тихо“;
            // на iPhone – файлът в приложението или в Library/Sounds (своите звуци)
            sound: notificationFileName(sound),
            // за Android под 8; на по-новите вибрацията е от канала
            vibrate: s.vibrate ? vibrationFor(use) : undefined,
            data: { sig, kind: n.kind, prayer: n.prayer, sound: n.sound, at: n.at.getTime() },
            ...(Platform.OS === 'android' ? { color: '#D4A857' } : {}),
        },
        trigger: {
            type: Notifications.SchedulableTriggerInputTypes.DATE,
            date: n.at,
            channelId: channel,
        },
    });
}

await cleanupChannels(new Set(wanted.map((w) => w.channel))).catch(() => {});

useNotificationStatus.setState({
    count: plan.length,
    until: plan.length ? plan[plan.length - 1].at : null,
    alarms: alarmCount,
});
}

/* ----- алармата (Android, етап 5) */

const isAlarm = (n: PlannedNotification) => n.kind === 'prayer' && n.sound === 'adhan' && n.prayer !== null;

/** Native алармата се ползва, ако я има в build-a и точните аларми не са забранени. */
function nativeAlarmsEnabled(exact: boolean | null): boolean {
    return Platform.OS === 'android' && hasNativeAlarms() && exact !== false;
}

/** Едно известие от плана → аларма за native частта, с всички текстове на езика на телефона. */
function toNativeAlarm(n: PlannedNotification, vibrate: boolean, place: string, sound: SoundDef):
NativeAlarm {
    const { lang, t } = getI18n();
    const prayer = (n.prayer ?? 'dhuhr') as PrayerId;
    const name = t.prayers[prayer];
    return {
        id: n.id,
        at: n.at.getTime(),
        prayer,
        title: t.alarm.title(name),
        arabic: PRAYERS[prayer].arabic,
        place,
        notifTitle: n.title,
        notifBody: n.body,
    };
}

```

```

    colors: [...PHASE_GRADIENTS[prayer]] as [string, string, string],
    vibrate,
    // вграден - името в res/raw; свой - пътят до файла
    sound: sound.file,
    lang,
    labels: {
      app: t.alarm.app,
      // „Спри езана“ - при езан; при мелодия или свой звук - „Спри алармата“
      stop: sound.category === 'adhan' || sound.category === 'takbir' ? t.alarm.stop : t.alarm.stopAlarm,
      mute: t.alarm.mute,
      muteShort: t.alarm.muteShort,
      close: t.alarm.close,
      soundName: sound.names[lang],
      channel: t.alarm.channel,
    },
  },
};
}

/** Трие всички известия на Езан и ги планира наново (от „Проверка на известията“). */
export async function resetAllNotifications(): Promise<void> {
  if (!SUPPORTED) return;
  await running;
  const existing = await Notifications.getAllScheduledNotificationsAsync();
  for (const r of existing) {
    if (r.identifier.startsWith('ezan-')) await
Notifications.cancelScheduledNotificationAsync(r.identifier);
  }
  created.clear();
  await rescheduleNotifications();
}

/**
 * Пробно известие след 10 сек. - със звука на известията. Връща часа, в който ще дойде.
 * (Звукът на алармата се преслушва направо в избора на звук - етап 6.)
 */
export async function sendTestNotification(): Promise<number> {
  const at = Date.now() + 10_000;
  if (!SUPPORTED) return at;
  const s = useSettings.getState();
  const sound = soundChoice().notify;
  const channel = await ensureChannel('notify', sound, s.vibrate);
  const { t } = getI18n();
  await Notifications.scheduleNotificationAsync({
    identifier: `test-${Date.now()}`,
    content: {
      title: t.notifications.testTitle,
      body: t.notifications.testBody,
      sound: notificationFileName(sound),
      vibrate: s.vibrate ? vibrationFor('notify') : undefined,
      data: { kind: 'test', at },
    },
  },
  trigger: {
    type: Notifications.SchedulableTriggerInputTypes.DATE,
    date: new Date(at),
    channelId: channel,
  },
});
  return at;
}

/** Как се показва известие, докато приложението е отворено: като обикновено, със звук. */

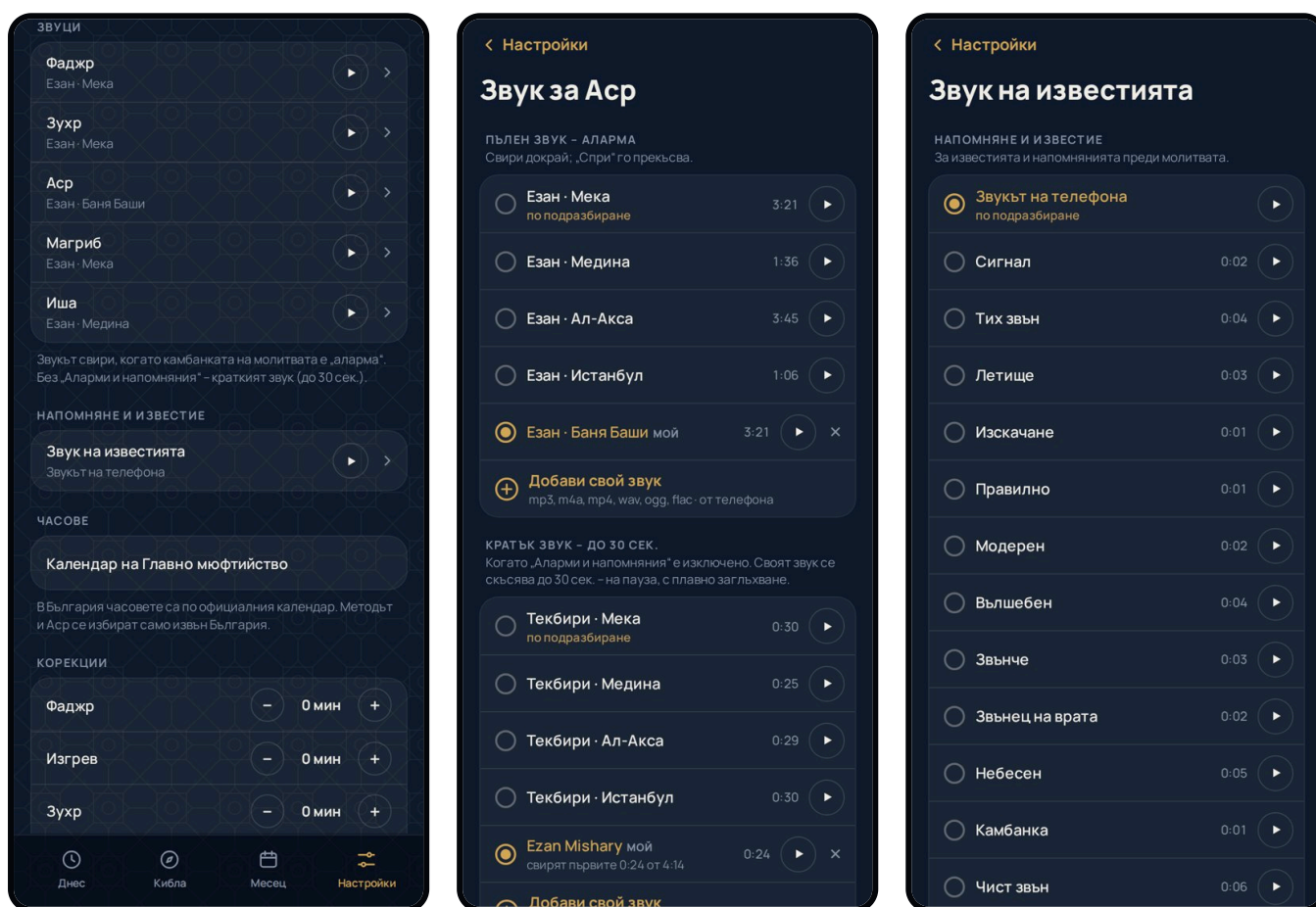
```

```
export function configureNotificationHandler(): void {
  if (!SUPPORTED) return;
  Notifications.setNotificationHandler({
    handleNotification: async () => ({
      shouldShowBanner: true,
      shouldShowList: true,
      shouldPlaySound: true,
      shouldSetBadge: false,
    }),
  });
}
```

Стъпка 7

Екраните

По одобрения mockup.



Настройки → Звуци · Звук за Аср · Звук на известията

SoundsGroup.tsx src/components/settings

- „Звуци“ в Настройки + жълтото поле „Пълният езан е изключен“ (Android без „Alarms & reminders“).

src/components/settings/SoundsGroup.tsx

133 реда

```
import { router } from 'expo-router';
import { Platform, Pressable, StyleSheet, Text, View } from 'react-native';

import { hasNativeAlarms } from '../../../modules/adhan-native';
```

```

import { ChevronIcon } from '@components/icons';
import { PlayButton } from '@components/sounds/PlayButton';
import { ALARM_PRAYERS, findSound, type SoundDef } from '@domain/sounds';
import { useI18n } from '@i18n';
import { openExactAlarmSettings, useNotificationStatus } from '@services/notifications';
import { useSounds } from '@store/sounds';
import { colors } from '@theme/colors';
import { fonts } from '@theme/typography';

import { Group } from './controls';

/**
 * „Звуци“ в Настройки (етап 6): звукът на всяка молитва с ► за преслушване
 * и звукът на известията. На Android се показва пълният звук на алармата,
 * на iPhone – краткият (до 30 сек.).
 */
export function SoundsGroup() {
  const { lang, t } = useI18n();
  const s = t.sounds;
  const { full, short, notify, custom } = useSounds();
  const exact = useNotificationStatus((x) => x.exact);
  const android = Platform.OS === 'android' && hasNativeAlarms();
  // Android без „Аларми и напомняния“ – свири само краткият звук
  const fullOff = android && exact === false;

  const soundFor = (p: (typeof ALARM_PRAYERS)[number]) =>
    android ? findSound(full[p], 'full', custom) : findSound(short[p], 'short', custom);

  return (
    <>
    <Group label={s.group} note={android ? s.noteAndroid : s.noteIos}>
      {ALARM_PRAYERS.map((p, i) => (
        <SoundRow
          key={p}
          first={i === 0}
          label={t.prayers[p]}
          sound={soundFor(p)}
          lang={lang}
          onPress={() => router.push({ pathname: '/sound', params: { p } })}
        />
      ))}
    </Group>
    {fullOff && (
      <View style={styles.warn}>
        <Text style={styles.warnIcon}>▲</Text>
        <View style={styles.warnText}>
          <Text style={styles.warnTitle}>{s.fullOffTitle}</Text>
          <Text style={styles.warnBody}>{s.fullOffText}</Text>
          <Pressable onPress={openExactAlarmSettings} accessibilityRole="button" style={styles.fix}>
            <Text style={styles.fixText}>{s.allow}</Text>
          </Pressable>
        </View>
      </View>
    )}
    <Group label={s.notifyGroup}>
      <SoundRow
        first
        label={s.notifyRow}
        sound={findSound(notify, 'short', custom, 'notify')}
        lang={lang}

```

```

        onPress={() => router.push({ pathname: '/sound', params: { p: 'notify' } })}
      />
    </Group>
  </>
);
}

function SoundRow({
  label,
  sound,
  lang,
  onPress,
  first,
}: {
  label: string;
  sound: SoundDef;
  lang: 'bg' | 'en';
  onPress: () => void;
  first?: boolean;
}) {
  return (
    <Pressable
      onPress={onPress}
      accessibilityRole="button"
      style={({ pressed }) => [styles.row, !first && styles.border, pressed && styles.pressed]}
    >
      <View style={styles.text}>
        <Text style={styles.label}>{label}</Text>
        <Text style={styles.sub} numberOfLines={1}>
          {sound.names[lang]}
        </Text>
      </View>
      <PlayButton sound={sound} />
      <ChevronIcon direction="right" size={16} color={colors.muted} />
    </Pressable>
  );
}

const styles = StyleSheet.create({
  row: { flexDirection: 'row', alignItems: 'center', gap: 10, paddingLeft: 14, paddingRight: 10,
paddingVertical: 9, minHeight: 56 },
  border: { borderTopWidth: StyleSheet.hairlineWidth, borderTopColor: 'rgba(255,255,255,0.10)' },
  pressed: { opacity: 0.7 },
  text: { flex: 1, gap: 2 },
  label: { fontFamily: fonts.semibold, fontSize: 14, color: colors.text },
  sub: { fontFamily: fonts.regular, fontSize: 12, color: colors.muted },
  warn: {
    flexDirection: 'row',
    gap: 12,
    padding: 14,
    borderRadius: 18,
    backgroundColor: 'rgba(227,154,75,0.12)',
    borderWidth: 1,
    borderColor: 'rgba(227,154,75,0.45)',
  },
  warnIcon: { fontSize: 18, color: colors.warn, marginTop: -1 },
  warnText: { flex: 1, gap: 2 },
  warnTitle: { fontFamily: fonts.bold, fontSize: 14, color: colors.warn },
  warnBody: { fontFamily: fonts.regular, fontSize: 12.5, lineHeight: 17, color: colors.textDim },
  fix: {
    alignSelf: 'flex-start',
  },
});

```

```

    marginTop: 8,
    paddingHorizontal: 14,
    paddingVertical: 7,
    borderRadius: 999,
    backgroundColor: colors.warn,
  },
  fixText: { fontFamily: fonts.extrabold, fontSize: 13, color: '#1F1206' },
});

```

sound.tsx src/app

- Изборът на звук: секции, ►, „по подразбиране“ най-отгоре, „МОЙ“ и ✕, „+ Добави свой звук“, индикаторът при рязане, „Същият звук за всички молитви“, съобщенията.

src/app/sound.tsx

344 реда

```

import { router, useLocalSearchParams } from 'expo-router';
import { useEffect, useRef, useState } from 'react';
import { ActivityIndicator, Alert, Platform, Pressable, ScrollView, StyleSheet, Text, View } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';
import Svg, { Path } from 'react-native-svg';

import { GeometricPattern } from '@components/GeometricPattern';
import { ChevronIcon } from '@components/icons';
import { PlayButton } from '@components/sounds/PlayButton';
import { Toast } from '@components/Toast';
import { PRAYER_IDS, type PrayerId } from '@domain/prayers';
import {
  BUILTIN_SOUNDS,
  customToDef,
  DEFAULT_FULL,
  DEFAULT_NOTIFY,
  DEFAULT_SHORT,
  findSound,
  fitsUse,
  formatDuration,
  type SoundDef,
  type SoundKind,
  type SoundUse,
} from '@domain/sounds';
import { useI18n } from '@i18n';
import { addCustomSound, removeCustomSound, stopPreview } from '@services/sounds';
import { useSounds } from '@store/sounds';
import { colors } from '@theme/colors';
import { fonts, tabularNums } from '@theme/typography';

/**
 * Изборът на звук (етап 6) – за една молитва (`p=asr`) или за известията (`p=notify`).
 * Android: пълен звук за алармата (вградени + свои + „Добави свой звук“) и кратък резервен.
 * iPhone: само кратки звуци до 30 сек. (вградени + свои + „Добави свой звук“).
 */
export default function SoundScreen() {
  const insets = useSafeAreaInsets();
  const { lang, t } = useI18n();
  const s = t.sounds;
  const { p } = useLocalSearchParams<{ p?: string }>();
  const prayer = PRAYER_IDS.includes(p as PrayerId) ? (p as PrayerId) : null;
  const store = useSounds();
  const android = Platform.OS === 'android';

```

```

const [busy, setBusy] = useState(false);
/** Файлът е избран и се обработва - показва се индикатор над целия екран. */
const [working, setWorking] = useState<{ kind: SoundKind; name: string; use: SoundUse } | null>(null);
const [toast, setToast] = useState<{ id: number; text: string } | null>(null);
const toastId = useRef(0);
const say = (text: string) => setToast({ id: ++toastId.current, text });

// при излизане от екрана преслушването спира
useEffect(() => stopPreview, []);

const closing = useRef(false);
const back = () => {
  if (closing.current) return;
  closing.current = true;
  if (router.canGoBack()) router.back();
  else router.replace('/settings');
};

// известията - само звънове; алармата - езан, текбири и своите звуци (без звъновете)
const builtin = (kind: SoundKind, use: SoundUse = 'alarm') =>
  BUILTIN_SOUNDS.filter((x) => x.kind === kind && fitsUse(x, use));
const custom = (kind: SoundKind, use: SoundUse = 'alarm') =>
  store.custom.filter((c) => c.kind === kind && (c.use ?? 'alarm') === use).map(customToDef);

const selected = (kind: SoundKind): string => {
  if (!prayer) return findSound(store.notify, 'short', store.custom, 'notify').id;
  return findSound(kind === 'full' ? store.full[prayer] : store.short[prayer], kind, store.custom).id;
};

const choose = (kind: SoundKind, id: string) => {
  if (!prayer) store.setNotify(id);
  else store.setSound(kind, prayer, id);
};

const add = async (kind: SoundKind, use: SoundUse = 'alarm') => {
  setBusy(true);
  const r = await addCustomSound(kind, (name) => setWorking({ kind, name, use }), use);
  setWorking(null);
  setBusy(false);
  if (r.ok) {
    choose(kind, r.sound.id);
    say(
      r.sound.originalSec
        ? s.addedTrimmed(r.sound.name, formatDuration(r.sound.durationSec))
        : s.added(r.sound.name),
    );
    return;
  }
  if (r.reason === 'canceled') return;
  const name = r.name ?? '';
  say(
    r.reason === 'tooLong'
      ? use === 'notify'
        ? s.tooLongNotify(name, formatDuration(r.durationSec ?? 0))
        : s.tooLongFull(name)
      : r.reason === 'tooBig'
        ? s.tooBig
        : r.reason === 'unsupported'
          ? s.unsupported
          : s.unreadable,
  );
};

```

```

const remove = (sound: SoundDef) => {
  const c = store.custom.find((x) => x.id === sound.id);
  if (!c) return;
  Alert.alert(s.deleteTitle(c.name), s.deleteText, [
    { text: s.cancel, style: 'cancel' },
    { text: s.delete, style: 'destructive', onPress: () => removeCustomSound(c) },
  ]);
};

const list = (kind: SoundKind, all: SoundDef[], defaultId: string, addKind?: SoundKind, addUse: SoundUse
= 'alarm') => {
  const sel = selected(kind);
  // звукът по подразбиране – най-отгоре
  const items = [...all.filter((x) => x.id === defaultId), ...all.filter((x) => x.id !== defaultId)];
  return (
    <View style={styles.card}>
      {items.map((x, i) => {
        const on = x.id === sel;
        return (
          <Pressable
            key={x.id}
            onPress={() => choose(kind, x.id)}
            accessibilityRole="radio"
            accessibilityState={{ checked: on }}
            style={({ pressed }) => [styles.row, i > 0 && styles.border, pressed && styles.pressed]}
          >
            <View style={[styles.radio, on && styles.radioOn]}>{on && <View style={styles.dot} />}</View>
            <View style={styles.text}>
              <Text style={[styles.name, on && styles.nameOn]} numberOfLines={2}>
                {x.names[lang]}
                {x.custom ? <Text style={styles.mine}>` ${s.mine}`</Text> : null}
              </Text>
              {x.id === defaultId && <Text style={styles.byDefault}>{s.byDefault}</Text>}
              {x.originalSec ? (
                <Text style={styles.hint}>{s.trimmed(formatDuration(x.durationSec),
formatDuration(x.originalSec))}</Text>
              ) : null}
            </View>
            {x.durationSec > 0 && <Text style={styles.dur}>{formatDuration(x.durationSec)}</Text>}
            {!(x.id === 'system' && Platform.OS === 'ios') && <PlayButton sound={x} />}
            {x.custom && (
              <Pressable onPress={() => remove(x)} hitSlop={8} accessibilityRole="button"
accessibilityLabel={s.delete}>
                <Text style={styles.del}>x</Text>
              </Pressable>
            )}
          </Pressable>
        );
      })}
      {addKind && (
        <Pressable
          onPress={() => add(addKind, addUse)}
          disabled={busy}
          accessibilityRole="button"
          style={({ pressed }) => [styles.row, items.length > 0 && styles.border, pressed &&
styles.pressed]}
        >
          {busy ? (
            <ActivityIndicator size="small" color={colors.gold} style={styles.plusBox} />
          ) : (

```

```

        <View style={styles.plusBox}>
            <Svg width={20} height={20} viewBox="0 0 20 20">
                <Path d="M10 5v10M5 10h10" stroke={colors.gold} strokeWidth={2} strokeLinecap="round" />
            </Svg>
        </View>
    )}
    <View style={styles.text}>
        <Text style={styles.addText}>{busy ? s.adding : s.add}</Text>
        <Text style={styles.hint}>
            {addUse === 'notify' ? s.addNotifyHint : addKind === 'full' ? s.addFullHint :
s.addShortHint}
        </Text>
    </View>
</Pressable>
    )}
</View>
);
};

const section = (title: string, hint: string) => (
    <View style={styles.sectionHead}>
        <Text style={styles.sectionLabel}>{title.toUpperCase()}</Text>
        <Text style={styles.sectionHint}>{hint}</Text>
    </View>
);

const title = prayer ? s.title(t.prayers[prayer]) : s.notifyTitle;

return (
    <View style={styles.root}>
        <GeometricPattern />
        <View style={[styles.header, { paddingTop: insets.top + 8 }]}>
            <Pressable onPress={back} hitSlop={10} accessibilityRole="button" style={styles.backBtn}>
                <ChevronIcon direction="left" size={18} color={colors.gold} />
                <Text style={styles.backText}>{t.settings.title}</Text>
            </Pressable>
        </View>
        <ScrollView
            contentContainerStyle={[styles.content, { paddingBottom: insets.bottom + 40 }]}
            showsVerticalScrollIndicator={false}
        >
            <Text style={styles.title}>{title}</Text>

            {!prayer ? (
                <>
                    {section(s.notifyGroup, s.notifyHint)}
                    {list('short', [...builtin('short', 'notify'), ...custom('short', 'notify')], DEFAULT_NOTIFY,
'short', 'notify'))}
                </>
            ) : android ? (
                <>
                    {section(s.fullSection, s.fullHint)}
                    {list('full', [...builtin('full'), ...custom('full')], DEFAULT_FULL, 'full')}
                    {section(s.shortSection, s.shortHintAndroid)}
                    {list('short', [...builtin('short'), ...custom('short')], DEFAULT_SHORT, 'short')}
                </>
            ) : (
                <>
                    {section(s.iosSection, s.iosHint)}
                    {list('short', [...builtin('short'), ...custom('short')], DEFAULT_SHORT, 'short')}
                </>
            )
        }
    </View>
);

```

```

    })

    {prayer && (
      <Pressable
        onPress={() => {
          store.applyToAll(prayer);
          say(s.applied);
        }}
        accessibilityRole="button"
        style={({ pressed }) => [styles.applyAll, pressed && styles.pressed]}
      >
        <Text style={styles.applyText}>{s.applyAll}</Text>
        <Text style={styles.applyLink}>{s.apply}</Text>
      </Pressable>
    )}
    {prayer && Platform.OS === 'ios' && <Text style={styles.footnote}>{s.iosFuture}</Text>}
  </ScrollView>
  {working && (
    <View style={styles.overlay} accessibilityLiveRegion="polite">
      <View style={styles.overlayCard}>
        <ActivityIndicator size="large" color={colors.gold} />
        <Text style={styles.overlayTitle}>{working.kind === 'short' && working.use === 'alarm' ?
s.workingShort : s.workingFull}</Text>
        <Text style={styles.overlayName} numberOfLines={2}>
          {working.name}
        </Text>
        {working.kind === 'short' && working.use === 'alarm' && <Text style={styles.overlayHint}>
{s.workingHint}</Text>}
      </View>
    </View>
  )}
  <Toast message={toast} bottom={insets.bottom + 24} />
</View>
);
}

const styles = StyleSheet.create({
  root: { flex: 1, backgroundColor: colors.base },
  header: { paddingHorizontal: 16, paddingBottom: 4 },
  backBtn: { flexDirection: 'row', alignItems: 'center', gap: 4, paddingVertical: 6, alignSelf: 'flex-
start' },
  backText: { fontFamily: fonts.bold, fontSize: 15, color: colors.gold },
  content: { paddingHorizontal: 16, gap: 8 },
  title: { fontFamily: fonts.extrabold, fontSize: 26, color: colors.text, marginTop: 4, marginBottom: 4 },
  sectionHead: { marginTop: 10, paddingHorizontal: 6, gap: 2 },
  sectionLabel: { fontFamily: fonts.bold, fontSize: 11, letterSpacing: 1.1, color: colors.muted },
  sectionHint: { fontFamily: fonts.regular, fontSize: 12, lineHeight: 16, color: colors.muted },
  card: {
    backgroundColor: 'rgba(255,255,255,0.05)',
    borderWidth: 1,
    borderColor: 'rgba(255,255,255,0.06)',
    borderRadius: 18,
    overflow: 'hidden',
  },
  row: { flexDirection: 'row', alignItems: 'center', gap: 10, paddingLeft: 14, paddingRight: 10,
paddingVertical: 9, minHeight: 54 },
  border: { borderTopWidth: StyleSheet.hairlineWidth, borderTopColor: 'rgba(255,255,255,0.10)' },
  pressed: { opacity: 0.7 },
  radio: {
    width: 20,
    height: 20,

```

```

borderRadius: 10,
borderWidth: 2,
borderColor: 'rgba(242,239,232,0.35)',
alignItems: 'center',
justifyContent: 'center',
},
radioOn: { borderColor: colors.gold },
dot: { width: 10, height: 10, borderRadius: 5, backgroundColor: colors.gold },
text: { flex: 1, gap: 1 },
name: { fontFamily: fonts.semibold, fontSize: 14.5, color: colors.text },
nameOn: { color: colors.gold },
mine: { fontFamily: fonts.extrabold, fontSize: 10, color: colors.muted },
byDefault: { fontFamily: fonts.semibold, fontSize: 11.5, color: '#B8955A' },
dur: { fontFamily: fonts.medium, fontSize: 12, color: colors.muted, ...tabularNums },
del: { fontSize: 15, color: colors.muted, paddingHorizontal: 2 },
plusBox: {
  width: 22,
  height: 22,
  borderRadius: 11,
  borderWidth: 2,
  borderColor: colors.gold,
  alignItems: 'center',
  justifyContent: 'center',
},
addText: { fontFamily: fonts.bold, fontSize: 14.5, color: colors.gold },
hint: { fontFamily: fonts.regular, fontSize: 11.5, color: colors.muted },
applyAll: {
  marginTop: 12,
  flexDirection: 'row',
  alignItems: 'center',
  justifyContent: 'space-between',
  paddingHorizontal: 14,
  paddingVertical: 13,
  borderRadius: 16,
  backgroundColor: 'rgba(255,255,255,0.05)',
  borderWidth: 1,
  borderColor: 'rgba(255,255,255,0.07)',
},
applyText: { fontFamily: fonts.semibold, fontSize: 14, color: colors.text, flex: 1 },
applyLink: { fontFamily: fonts.bold, fontSize: 14, color: colors.gold },
overlay: {
  position: 'absolute',
  top: 0,
  left: 0,
  right: 0,
  bottom: 0,
  backgroundColor: 'rgba(6,10,20,0.72)',
  alignItems: 'center',
  justifyContent: 'center',
  padding: 32,
},
overlayCard: {
  alignSelf: 'stretch',
  alignItems: 'center',
  gap: 10,
  paddingVertical: 26,
  paddingHorizontal: 20,
  borderRadius: 22,
  backgroundColor: '#16243A',
  borderWidth: 1,
  borderColor: 'rgba(212,168,87,0.35)',

```

```

    },
    overlayTitle: { fontFamily: fonts.bold, fontSize: 16, color: colors.text, textAlign: 'center', marginTop:
4 },
    overlayName: { fontFamily: fonts.semibold, fontSize: 13.5, color: colors.gold, textAlign: 'center' },
    overlayHint: { fontFamily: fonts.regular, fontSize: 12.5, lineHeight: 17, color: colors.muted, textAlign:
'center' },
    footnote: { fontFamily: fonts.regular, fontSize: 12, lineHeight: 17, color: colors.muted, marginTop: 8,
paddingHorizontal: 6 },
  });

```

PlayButton.tsx src/components/sounds

- Кръглият бутон ▶ / ||.

src/components/sounds/PlayButton.tsx

48 реда

```

import { Pressable, StyleSheet } from 'react-native';
import Svg, { Path, Rect } from 'react-native-svg';

import type { SoundDef } from '@domain/sounds';
import { useI18n } from '@i18n';
import { togglePreview, usePreview } from '@services/sounds';
import { colors } from '@theme/colors';

/** Кръглият бутон ▶ / || - преслушване на звука. */
export function PlayButton({ sound }: { sound: SoundDef }) {
  const { lang, t } = useI18n();
  const playing = usePreview((s) => s.playing === sound.id);
  const name = sound.names[lang];
  return (
    <Pressable
      onPress={() => togglePreview(sound)}
      hitSlop={6}
      accessibilityRole="button"
      accessibilityLabel={playing ? t.sounds.stop(name) : t.sounds.play(name)}
      style={({ pressed }) => [styles.btn, playing && styles.on, pressed && styles.pressed]}
    >
      <Svg width={13} height={13} viewBox="0 0 24 24">
        {playing ? (
          <>
            <Rect x={5} y={4} width={5} height={16} rx={1.5} fill={colors.goldInk} />
            <Rect x={14} y={4} width={5} height={16} rx={1.5} fill={colors.goldInk} />
          </>
        ) : (
          <Path d="M7 4.5v15l13-7.5z" fill={colors.text} />
        )}
      </Svg>
    </Pressable>
  );
}

const styles = StyleSheet.create({
  btn: {
    width: 34,
    height: 34,
    borderRadius: 17,
    borderWidth: 1,
    borderColor: 'rgba(255,255,255,0.16)',
    alignItems: 'center',
    justifyContent: 'center',
  },

```

```

},
on: { backgroundColor: colors.gold, borderColor: colors.gold },
pressed: { opacity: 0.7 },
});

```

Стъпка 8

Тестове

218 теста общо; 17 са за звуците.

sounds.test.ts src/domain/__tests__

- Каталогът: кратките са до 30 сек., имената стават за Android.
- Изборът: свой звук, изтрит свой звук, пълен ≠ кратък, звук за аларма ≠ звук за известия, файлът за известието.
- Проверката на своя звук: 10 сек. / 30 сек. / 15 мин. / 60 MB, празен файл.
- Звуците за известия: до 10 сек., валидни имена, вписани в `app.json` и файлът съществува.

src/domain/__tests__/sounds.test.ts

140 реда

```

import {
  NOTIFY_MAX_SEC,
  allSounds,
  BUILTIN_SOUNDS,
  DEFAULT_FULL,
  DEFAULT_SHORT,
  findSound,
  formatDuration,
  nameFromFile,
  notificationFileName,
  validateUpload,
  type CustomSound,
} from '../sounds';

const mine: CustomSound = {
  id: 'custom_1',
  kind: 'full',
  name: 'Ezan Bania Bashi',
  file: 'file:///data/user/0/com.ilkoadamov.adhan/files/sounds/custom_1.mp3',
  durationSec: 201,
};

const mineIos: CustomSound = { id: 'custom_2', kind: 'short', name: 'Tekbir', file: 'custom_2.caf',
durationSec: 26 };

describe('каталогът на звуците', () => {
  it('има пълни и кратки звуци; кратките са до 30 сек.', () => {
    expect(BUILTIN_SOUNDS.some((s) => s.kind === 'full')).toBe(true);
    for (const s of BUILTIN_SOUNDS.filter((x) => x.kind === 'short'))
    expect(s.durationSec).toBeLessThanOrEqual(30);
  });

  it('id-тата са уникални, а имената на файловете стават за Android (a-z, 0-9, _)', () => {
    const ids = BUILTIN_SOUNDS.map((s) => s.id);
    expect(new Set(ids).size).toBe(ids.length);
    for (const s of BUILTIN_SOUNDS) expect(s.file).toMatch(/^[a-z0-9_]+$/);
  });
});

```

```

it('звучите по подразбиране ги има', () => {
  expect(findSound(DEFAULT_FULL, 'full', []).id).toBe(DEFAULT_FULL);
  expect(findSound(DEFAULT_SHORT, 'short', []).id).toBe(DEFAULT_SHORT);
});
});

describe('избраният звук', () => {
  it('своят звук се намира', () => {
    expect(findSound('custom_1', 'full', [mine])).toBe('Ezan Bania Bashi');
    expect(allSounds([mine, mineIos])).toHaveLength(BUILTIN_SOUNDS.length + 2);
  });

  it('изтрит свой звук → звукът по подразбиране', () => {
    expect(findSound('custom_1', 'full', []).id).toBe(DEFAULT_FULL);
  });

  it('пълнен звук не може да е кратък (и обратно)', () => {
    expect(findSound('adhan_madinah', 'short', []).id).toBe(DEFAULT_SHORT);
    expect(findSound('takbir_makkah', 'full', []).id).toBe(DEFAULT_FULL);
  });

  it('известията – само звънове; алармата – без звънове', () => {
    expect(findSound('chime_soft', 'short', [], 'notify').id).toBe('chime_soft');
    expect(findSound('takbir_makkah', 'short', [], 'notify').id).toBe('system');
    expect(findSound('chime_soft', 'short', []).id).toBe(DEFAULT_SHORT);
    expect(findSound('takbir_aqsa', 'short', []).id).toBe('takbir_aqsa');
    // премахнатите мелодии → звукът по подразбиране
    expect(findSound('melody_nur_short', 'short', []).id).toBe(DEFAULT_SHORT);
  });

  it('звукът на телефона – само за известията, „default“ за expo-notifications', () => {
    const sys = findSound('system', 'short', [], 'notify');
    expect(sys.id).toBe('system');
    expect(notificationFileName(sys)).toBe('default');
    expect(findSound('system', 'short', []).id).toBe(DEFAULT_SHORT);
  });

  it('своят звук за известия е само в списъка на известията', () => {
    const n: CustomSound = { id: 'custom_9', kind: 'short', name: 'Ding', file: 'content://x', durationSec:
3, use: 'notify' };
    expect(findSound('custom_9', 'short', [n], 'notify').id).toBe('custom_9');
    expect(findSound('custom_9', 'short', [n]).id).toBe(DEFAULT_SHORT);
    expect(findSound('custom_2', 'short', [mineIos], 'notify').id).toBe('system');
  });

  it('файлт за известието: вграден – с разширение; свой на iPhone – .caf', () => {
    expect(notificationFileName(findSound('chime_soft', 'short', [], 'notify'))).toBe('chime_soft.wav');
    expect(notificationFileName(findSound('custom_2', 'short', [mineIos]))).toBe('custom_2.caf');
  });
});

describe('свой звук – проверка', () => {
  it('на iPhone – до 30 сек.', () => {
    expect(validateUpload('short', 29.9, 500_000)).toBeNull();
    expect(validateUpload('short', 30.4, 500_000)).toBeNull(); // закръгляне на дължината
    expect(validateUpload('short', 222, 3_000_000)).toBe('tooLong');
  });

  it('за известия – до 10 сек.; по-дълъг не се добавя', () => {
    expect(validateUpload('short', 9.8, 100_000, 'notify')).toBeNull();
    expect(validateUpload('short', 12, 100_000, 'notify')).toBe('tooLong');
  });
});

```

```

});

it('на Android - до 15 мин. и 60 MB', () => {
  expect(validateUpload('full', 222, 3_000_000)).toBeNull();
  expect(validateUpload('full', 16 * 60, 3_000_000)).toBe('tooLong');
  expect(validateUpload('full', 222, 80 * 1024 * 1024)).toBe('tooBig');
});

it('празен или нечетим файл', () => {
  expect(validateUpload('full', -1, 1000)).toBe('empty');
  expect(validateUpload('short', 0, 1000)).toBe('empty');
});

it('името от файла и дължината', () => {
  expect(nameFromFile('Ezan_Bania-Bashi.mp3')).toBe('Ezan Bania-Bashi');
  expect(nameFromFile('.mp3')).toBe('Sound');
  expect(formatDuration(222)).toBe('3:42');
  expect(formatDuration(4.2)).toBe('0:04');
});
});

describe('вградените звуци за известията', () => {
  const app = require('../../app.json');
  const plugin = app.expo.plugins.find((p: unknown) => Array.isArray(p) && p[0] === 'expo-notifications');
  const listed: string[] = plugin[1].sounds;
  // eslint-disable-next-line @typescript-eslint/no-require-imports
  const fs = require('fs');

  it('всеки е до 10 сек., с валидно име за Android и е в app.json', () => {
    const notify = BUILTIN_SOUNDS.filter((s) => s.kind === 'short' && s.category === 'chime' && s.id !== 'system');
    expect(notify.length).toBe(22);
    for (const s of notify) {
      expect(s.durationSec).toBeLessThanOrEqual(NOTIFY_MAX_SEC);
      expect(s.file).toMatch(/^[a-z][a-z0-9_]*$/);
      const rel = `./assets/sounds/${s.file}.${s.ext}`;
      expect(listed).toContain(rel);
      expect(fs.existsSync(`${process.cwd()}/${rel}`)).toBe(true);
    }
  });

  it('id-тата са уникални', () => {
    const ids = BUILTIN_SOUNDS.map((s) => s.id);
    expect(new Set(ids).size).toBe(ids.length);
  });
});

```

Трябва да видиш: Tests: 218 passed, 218 total

npm test

Стъпка 9

Проверка и какво остава

На телефона:

- Настройки → Звуци: петте молитви и „Звук на известията“; ► пуска, втори тап спира.
- Тап на молитва → избери „Езан · Медина“ → в часа на алармата свири той, под вълната пише името му.

- „+ Добави свой звук“ в „Кратък звук“ → дълъг mp3 → „Изрязване на откъса...“ → появява се с „МОЙ“ и „свирят първите 0:24 от 4:14“.
- „Звук на известията“: „Звукът на телефона“ е най-отгоре и избран; свой звук над 10 сек. не се добавя.
- Изключи Settings → Apps → Adhan → Alarms & reminders → в Звуци се появява жълтото поле; в часа – краткият звук като известие.

Преди публикуване (етап 11): да се потвърди, че записите от PrayTimes.org (Мека, Медина, Ал-Акса) могат да се ползват в приложение, и откъде е записът „Истанбул“. Звуците за известия са от Pixabay – лицензът позволява.

Терминал

```
git add .
git commit -m "Етап 6: звуци"
git tag stage-6
git push
git push origin stage-6
```